



Iodine Spectrometer

Abstract

The aim of this experiment was to analyse the band structure of the diatomic molecule iodine by determining molecular constants such as the fundamental vibrational frequency, the anharmonicity constant and the dissociation energy, which were then compared with accepted literature results falling within reasonable agreement. The light of a tungsten lamp transmitted through a cell containing iodine vapour was analysed obtaining its spectrum and ultimately the Birge-Sponer plot from which the molecular constants ($\omega_e = 128.3 \pm 1.9 \text{ cm}^{-1}$, $\omega'_e X'_e = 0.908 \pm 0.039 \text{ cm}^{-1}$, $D'_e = 4497 \pm 213 \text{ cm}^{-1}$) pertaining the electronic state B were extracted from. A good linear fit was obtained for the Birge-Sponer plot showcasing the linear relationship between the first differences of the bands and their vibrational quantum numbers v' . The B band of the molecule was successfully studied and its Morse potential as a function of internuclear radius was obtained allowing thorough visualization of the molecule's vibrational structure.

Contents

1	Introduction	2
2	Theory	2
2.1	Molecular bands and spectral structure of I_2	2
2.2	Vibrational structure and numerical determination of the dissociation energy	4
2.3	The Birge-Sponer plot	4
3	Methodology	5
3.1	Apparatus and set up	5
3.2	Experimental procedure	6
3.2.1	Calibration	6
3.2.2	Iodine molecular spectrum and the Birge-Sponer plot	6
4	Results and discussion	7
4.1	Visualization of the iodine spectrum	7
4.2	The Birge-Sponer plot	8
4.3	The Morse potential	9
5	Conclusion	10

1 Introduction

Molecular spectra arise from transitions between rotational, vibrational, and electronic energy levels[1]. When molecules absorb or emit radiation, simultaneous changes in these levels occur, producing characteristic band systems in the ultraviolet and visible regions[1]. Iodine is a particularly useful case: its long absorption series contains bands corresponding to transitions from the electronic ground state X to the excited state B, yielding a rich and accessible band spectrum in the visible range (400–650 nm)[2].

The aim of this experiment was to analyse the vibrational structure of iodine by determining molecular constants such as the fundamental vibrational frequency, the anharmonicity constant, the dissociation energy, and the force constant, which were then compared the with accepted literature results.

This was done by passing the continuous light of a tungsten lamp through a glass cell containing iodine vapour after which, the transmitted spectrum was recorded using a *Baird & Watts Monospek 1000* spectrometer. After calibration, the absorption bands were analysed and a plot of the first differences $\Delta^1 G(v)$ against the vibrational quantum number v was constructed, from which the molecular constants of iodine were extracted.

2 Theory

2.1 Molecular bands and spectral structure of I_2

There exists three types of band systems in molecules: rotational (due to the rotation of

the molecules around its centre), vibrational (due to the vibration of the atoms composing the molecule) and electronic (due to the electronic transitions within the molecule)[1]. Absorption or emission of radiation by the molecule results in changes on their rotational, vibrational and/or electronic energies which can be represented in the molecule's spectrum[1]. Due to the large amount of energy involved in electronic transitions, these are often accompanied by a change the molecule's rotation and vibrational bands leading to broader absorption or emission bands[1].

Figure 1 shows a plot of potential energy as a function of interatomic distance r . Curves X, A and B represent the different electronic states with X usually representing the ground state and A, B, C... representing the excited states in order of increasing energy[3]. The A state results in a spectrum in the red side of the visible region which will not be used in this experiment[3]. Instead, states X and B will be the ones studied.

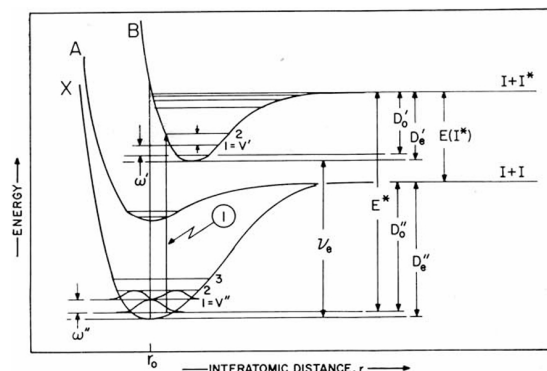


Figure 1: Representation of molecular band structure - potential energy as a function of internuclear radius r [3]

Vibrational levels, represented as $v = 0, 1, 2, \dots$ are associated to the characteristic vibrational energies ω which are grossly exaggerated in the diagram[3]. Notation like v' and v'' or ω' and ω'' might be used to dif-

ferentiate between the parameters associated with the X and B states respectively.

D_0 indicates the dissociation energy of the state whereas D_e represents the *extrapolated* dissociation energy [3]. This is the energy at which a molecule reaches its dissociation limit and at which the spectral bands disappear[1]. Past this point, the energy differences amongst vibrational levels is zero[1].

A transition between two different vibrational levels belonging to different electronic states are represented as vertical lines (arrow 1 in Fig. 1). This transition can be written as $(B, v' = 2 \leftarrow X, v'' = 0)$ [3].

Each transition appears in the spectrum of the molecule as a "bump" forming, as a whole, an image like the one shown in Figure 2. Region (2) shows the wavelengths at which not only the vibrational series $v'' = 0$ are noticeable but also the ones corresponding to $v'' = 1$. Region (3) is where dissociation is

reached and the distance between vibrational levels is zero. Meanwhile, region (4) indicates the start of where transitions from the X state to the A state would be, transitions which will not be used in this experiment[3].

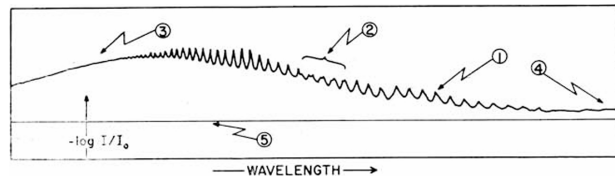


Figure 2: Spectrum of I_2 in air in the 400-650 nm range. (1) A band head. (2) Region where $(v', 1'')$ bands appear in the I_2 spectrum. (3) Region in which the convergence limit is located. (4) Start of the $(A \leftarrow X)$ near-infrared bands. (5) "Base" line.[3]

Figure 3 zooms into the central region of the spectrum. Where absorption lines caused by vibrational series $v'' = 1$ and $v'' = 2$ can be appreciated and what are referred to as sections (2), (3) and (4) 2 can be seen (note here the wavelength is decreasing left to right).

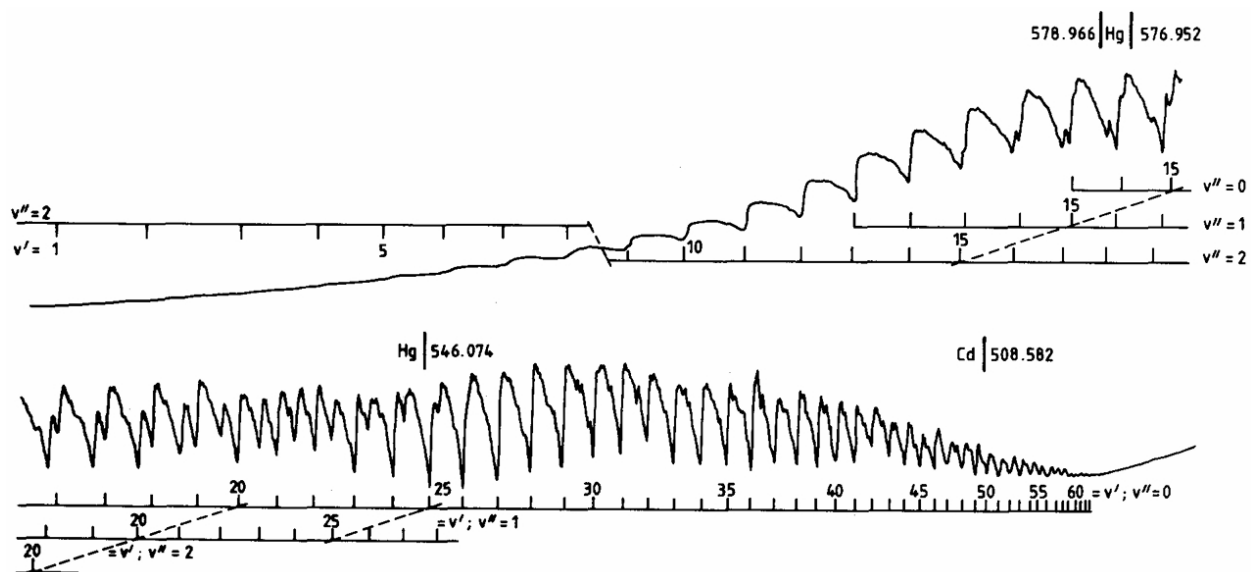


Figure 3: Iodine's visible absorption spectra along with reference lines from mercury and cadmium lamps as well as the three vibrational series $v'' = 0, 1, 2$ [2].

2.2 Vibrational structure and numerical determination of the dissociation energy

Rotational levels were not included in Figure 1 as their energy cannot be resolved with this experiment[3]. It is then safe to define the total energy of the molecule as

$$E = T_e + G(v) \quad (1)$$

where T_e and $G(v)$ are the electronic and vibrational energies respectively[1]. $G(v)$ can be written as follows:

$$G(v) = \omega_e(v+1/2) - \omega_e X_e(v+1/2)^2 + \dots \quad (2)$$

with ω_e and $\omega_e X_e$ being the fundamental vibrational frequency and the anharmonicity constant[1].

These parameters can be determined by examining the spacing between successive vibrational bands:

$$\Delta^1 G(v) = \Delta\nu \quad (3a)$$

$$\Delta^2 G(v) = \Delta^1 G(v) - \Delta^1 G(v+1) \quad (3b)$$

where $\Delta^1 G(v)$ and $\Delta^2 G(v)$ are respectively denominated the first and second differences[1]. $\Delta\nu$ on the other hand, is the difference in the radiation frequencies between transitions from v'' to v' levels when absorption occurs[1]. These frequencies can be defined as follows[1]

$$\nu = (T'_e - T''_e) + G(v) - G(0) \quad (4)$$

leading to

$$\Delta\nu = G(v+1) - G(v)$$

By plugging this and Equation 2 into Equation 3, the first and second differences can be rewritten as

$$\Delta^1 G(v) = \omega_e - 2(v+1)\omega_e X_e \quad (5a)$$

$$\Delta^2 G(v) = 2\omega_e X_e \quad (5b)$$

Once dissociation is reached, the spacing between band will be zero, i.e.

$$\omega'_e - 2(v'+1)\omega'_e X'_e = 0$$

resulting in the determination of the vibrational quantum number $v = v_m$ at which the dissociation limit is reached[1]:

$$v_m = (1/2X_e) - 1 \quad (6)$$

This allows the calculation of the dissociation energy[1]:

$$D_e = G(v_m) \quad (7)$$

2.3 The Birge-Sponer plot

The Birge-Sponer plot consists of plotting the first differences $\Delta^1 G(v)$ against the vibrational quantum number (v). This unveils a linear relationship between the two with a slope equal to twice the negative anharmonicity constant ($-2\omega_e X_e$) and an intercept at $v = -1$ equal to ω_e [1]. However, for large values of v ($v' > 50$ [2]) the cubic and quadratic terms of the $G(v)$ expansion make the curve deviate from the linear relationship making the decrease in $\Delta^1 G(v)$ steeper as v increases, just as show in Figure 4[1]. Once ω_e is obtained, the force constant can be calculated as

$$k_e = 4\pi^2 c^2 \mu \omega_e \quad (8)$$

where c is the speed of light and μ is the reduced mass of the molecule[1].

The straight line can be extrapolated to find the vibrational quantum number $v = v_m$ and the dissociation energy D_e as the area under the curve[1].

Once these molecular constants have been obtained, the Morse potential for the given electronic state can be obtained as

$$U(r - r_e) = D_e(1 - e^{-\beta(r-r_e)})^2 \quad (9)$$

$$\beta = \omega_e(2\mu c/D_e h)^{1/2}$$

where r_e is the equilibrium internuclear distance[1]. $r_e = 3.016 \times 10^{-8}$ cm for iodine's B electronic state and $r_e = 2.667$ for its X state[4].

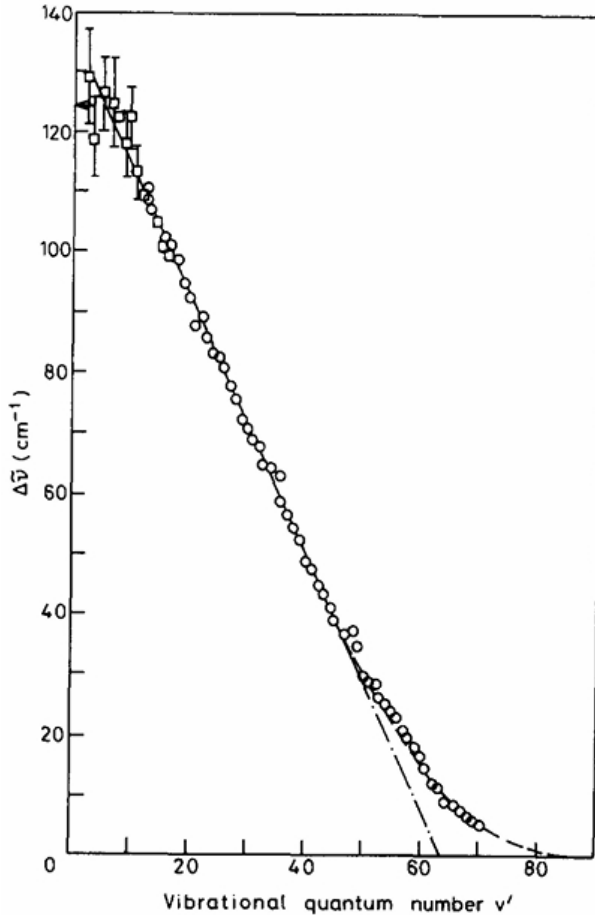


Figure 4: Birge-Sponer plot of the B-X system of molecular iodine with the differences $\Delta\tilde{\nu} = \Delta^1G(v')$ being plotted in cm^{-1} against the vibrational quantum number v' [2].

3 Methodology

3.1 Apparatus and set up

The apparatus, as seen in Figure 5, consisted on a *Baird & Watts Monospek 1000* powered by a high-voltage power supply (HVPS) and with a motor attached to run through the wavelength region set to scan. Aligned in front of the entrance slit of the spectrometer there was positioned the source light needed for the each part of the experiment that was being performed at the given moment. On the other side, attached to the exit slit, there was a photomultiplier tube (PMT) attached to the exit slit converting the light into readable voltage signal which was detected by the NI USB-6008 and transferred to the computer for the analysis of the data.

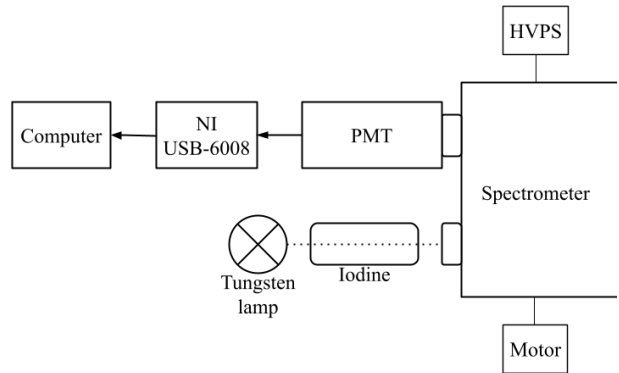


Figure 5: Schematic of the apparatus set-up. With a *Baird & Watts Monospek 1000* spectrometer, a motor, a high-voltage power supply (HVPS), a tungsten lamp and an iodine cell, a photomultiplier tube (PMT) and a National Instruments USB-6008/6009 connected to the computer.

The sources used for calibration were: a red He-Ne laser, a mercury lamp and a sodium lamp. Once the laser was aligned with the entrance slit, leaving enough space between the two, the laser was used to position and align the rest of the sources.

3.2 Experimental procedure

3.2.1 Calibration

The widths of the entrance and exit slits along with the speed of the motor were adjusted until a good enough signal in the 400-650 nm range could be obtained. After this, the calibration was started; recording the spectra of the calibration sources from 650 through 400 nm and cross-matching the observed peak positions of the emission lines with their known wavelengths[5].

For each source, the voltage of the HVPS was adjusted accordingly so that a strong signal could be obtained without surpassing the maximum 10 V reading signal set by the NI USB-6008. In addition, since the experiment relies on turning on the motor and start the recording of the readings through the computer at the exact same time, the spectrum of each source was measured at least 5 times. Then an average of the bandhead positions was taken with the goal to improve the reproducibility of the experiment.

Once the positions of the bandheads of the spectra were acquired and linked to their wavelengths, a linear relationship $\lambda = (x - c)/m$ between the two was obtained. Where x is the position and the parameters m and c are the slope and intercept of the straight line. These were later used to convert into wavelengths the x-axis of the spectra.

3.2.2 Iodine molecular spectrum and the Birge-Sponer plot

The spectrum of the tungsten lamp with and without iodine was recorded multiple times since, again, the experiment relies on starting the motor and the recording of the readings at the same time. From these recordings, a pair of *with* and *without* iodine spectra with

very similar delay in the start of the recordings (and therefore very good overlapping) was selected for the experiment.

A wavelength offset was applied to cancel this delay. This was done by examining the obtained spectrum with iodine and comparing it to the spectrum shown in Figure 3, cross-matching the absorption lines to calculate the offset that was necessary to apply.

Once this was calculated and applied, $\log(I_0/I)$ was plotted against the wavelengths λ . Where I and I_0 represent respectively the signal for the spectrum of the tungsten line with and without iodine.

The peaks due to the vibrational series $v'' = 0$ were numbered according to Figure 3 and used to calculate the first differences $\Delta^1 G(v')$ in cm^{-1} which were plotted against the vibrational quantum numbers v' to create a Birge-Sponer plot.

From there, parameters like ω'_e , $\omega'_e X'_e$, v'_m , D'_e , D'_0 and k'_e were calculated as detailed in section 2.3. Numerical calculations of those constants were also performed for comparison following the equations in section 2.2. These were compared to literature values from Herzberg^[4] calculating the difference between them.

Finally, these parameters were used to calculate and plot a representation of the Morse potential as a function for internuclear distance r .

This process was attempted to be repeated for the transitions ($v' \leftarrow 1''$). However, the signal was not clear enough to detect and identify enough spectral bands belonging to this vibrational series to continue with the analysis (see Appendix).

4 Results and discussion

The calibration line shown in Figure 6 was obtained, showing the linear relationship between iterations and wavelength with a slope of -12.66 ± 0.11 and an intercept at 8231.70 ± 68.66 .

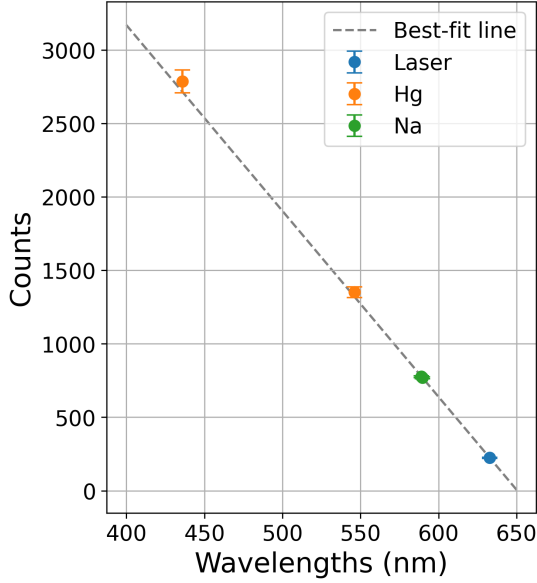


Figure 6: Calibration line for iterations to wavelengths conversion using an He-Ne red laser, a mercury and a sodium lamp.

It can be observed how the shorter the wavelengths, the bigger the errors. This is because the scan was made starting at 650 nm and decreasing in wavelength and the

longer the spectrometer had been running for, the larger the possibility of the spectrometer skipping some wavelengths will be. This was consistent throughout the rest of the experiment as all spectrums were measured starting at 650.0 nm.

Although more (but weaker) spectral lines could have been added into the calculation of the calibration line, it was considered that the line was good enough with 5 data points as it goes close to (if not through) the error window of all points involved and χ^2 's p-value for the linear fit was 0.644 indicating the fit is valid[6].

4.1 Visualization of the iodine spectrum

A wavelength offset of 9.91 nm was calculated and applied as described in section 3.2.2 with a statistical uncertainty of 0.19 nm (see Appendix for the process of comparison and determination of the offset). The semi-log plot obtained after the offset was applied can be seen in Figure 7 showing the regions described by Figure 2 including the convergence limit, where $(\nu', 1'')$ series is noticeable and where the near-infrared bands would start appearing.

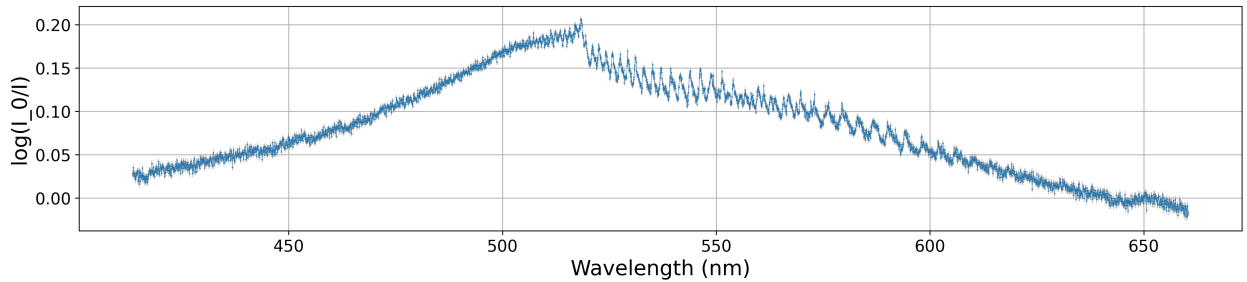


Figure 7: Semi-log plot obtained from plotting the log of the spectrum of the tungsten lamp against the the spectrum of the transmitted light of the tungsten lamp through the cell of iodine vapour.

4.2 The Birge-Sponer plot

With the absorption lines numbered and the first differences calculated, the Birge-Sponer plot seen below was obtained.

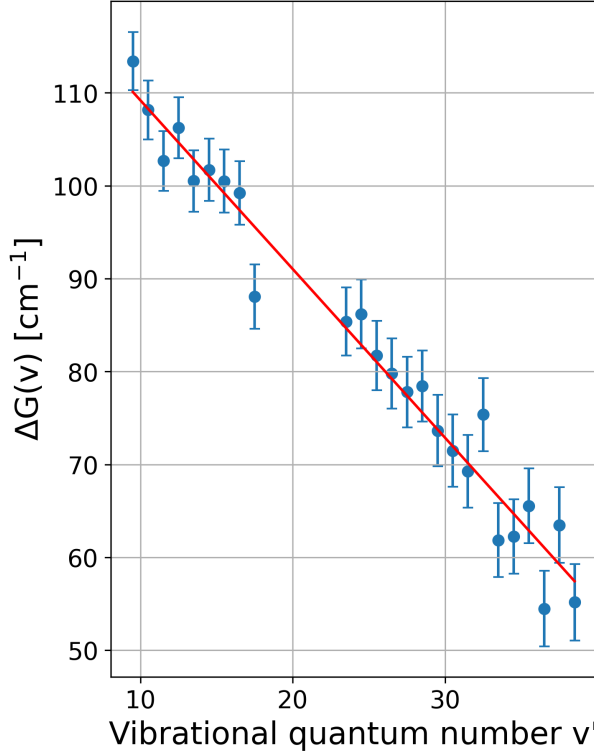


Figure 8: Birge-Sponer plot showcasing the linear relationship between the differences $\Delta^1G(v')$ and the vibrational quantum numbers v' for $v' < 50$.

In Figure 8 vibrational quantum numbers

between 9 and 40 show a linear relationship between the first difference and the vibrational quantum numbers where $\Delta^1G(v')$ decreases as v' increases. Bands past $v' = 40$ could not be accurately included without sacrificing the goodness of the fit as the bands started being less sharp and the errors started being greater (as they correspond to shorter wavelengths). Absorption lines representing $18 > v' > 23$ (corresponding to section 2 in Figure 2) were numbered but not plotted as it was difficult to distinguish them to the transitions corresponding to other vibrational series.

With the vibrational quantum numbers $v' > 40$ not being included, it was not possible to observe the deviation from linearity achieved around $v' > 50$ [2]. Despite this, a good linear fit[6] was obtained (χ^2 p-value of 0.683) and an accurate calculation of the molecular constants were achieved (see Table 1).

The deviation of the obtained values from the ones provided in literature[4] can be seen tabulated. The large difference on $\omega'_e X'_e$ might be due to its small value and therefore high sensitivity to noise, errors and peak misalignment.

	Graphical (cm ⁻¹)	Numerical (cm ⁻¹)	Accepted (cm ⁻¹) ^{[3][4]}	Difference (%)
ω'_e	128.3 ± 1.9	142.5 ± 1.4	128.0	0.23
$\omega'_e X'_e$	0.908 ± 0.039	1.214 ± 0.109	0.834	8.87
D'_e	4497 ± 213	4179 ± 377	4398	2.25
D'_0	4432 ± 213	4108 ± 377	4335	2.24

Table 1: Table containing results obtained through the Birge-Sponer plot, results obtained numerically and literature values from Herzberg[4] and Fred E. Stafford[3]. Last column indicates the difference in percentage between the graphical results and the accepted values.

It is also important to note that the values obtained through the Birge-Sponer plot were closer to the accepted literature values than those obtained through numerical methods and often with larger uncertainties. Again, this might be due to the propagation of errors and the higher sensitivity of these through numerical methods (specially the sensitivity to errors and deviation on the seconds differences $\Delta^2 G(v')$ - used in through all of the numerical calculations but none of the graphical ones) compared to the *smoothing* of the noise through the best fit line.

The force constant was calculated to be $k'_e = 6.146 \times 10^4 \pm 0.18 \times 10^4$ dyn/cm, which, although it does not agree, it is in a very close order of magnitude as the literature value of $k'_e = 1.72 \times 10^5$ by Herzberg[1][4]. The vibrational quantum number v_m was found to be $v'_m \approx 70 \pm 3$ which is lower than the last

bound value found by LeRoy and Bernstein of $v' = 87$ [2]. The difference in these values is due to the fact that the value $v'_m = 70$ was calculated by extrapolating the straight fit line and therefore didn't take into account the non-linearity for values $v' > 50$.

4.3 The Morse potential

With these constants having been calculated, the Morse potential for the electronic state B was calculated and plotted as it can be seen Figure 9. This can be compared to Figure 1 to analyse the true energies at the different vibrational quantum numbers $v' < 50$ (including $G(0')$), where the linearity is maintained. The decrease in the first differences as v' increases until dissociation is reached at D'_e can also be noted as well as the minimum in potential energy $U(r - r_e)$ at the equilibrium internuclear distance r_e .

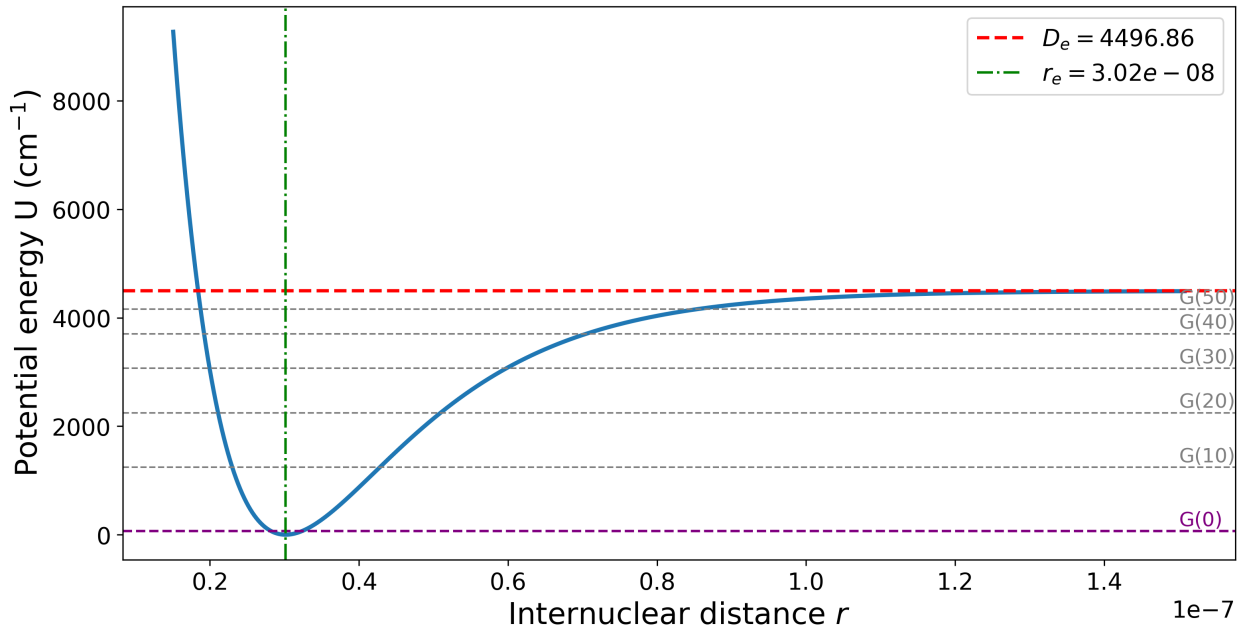


Figure 9: Morse potential plot of I_2 's electronic state B showcasing the dissociation energy as well as the vibrational energy for quantum numbers $v' < 50$ where linearity is conserved.

5 Conclusion

With the data obtained from this experiment, the band structure of the diatomic molecule iodine was successfully studied, observing a linear relationship between the first differences of the bands and the vibrational quantum numbers for $v < 50$ in the Birge-Sponer plot. A plot that showed a fundamental vibrational frequency of $128.3 \pm 1.9 \text{ cm}^{-1}$ and an anharmonicity constant of $0.908 \pm 0.039 \text{ cm}^{-1}$ as well as a dissociation energy D'_e of $4497 \pm 213 \text{ cm}^{-1}$ which can also be appreciated in the plot of the Morse potential obtained for the B electronic state. A force constant of $6.146 \times 10^4 \pm 0.18 \times 10^4 \text{ dyn/cm}$ was calculated as well as a vibrational quantum number $v' = v'_m \approx 70$ which was obtained by extrapolating the straight fit line from the Birge-Sponer plot, causing it to deviate from the last bound value $v' = 87$ found by LeRoy and Bernstein since the bandheads appearing after linearity breaks were not able to be identified. Other than that, all values fall within reasonable agreement with the literature data, confirming the reliability of the method despite its reproducibility.

Although uncertainties were included with each measurement, reproducibility was improved and goodness-of-fit tests were applied where applicable, several sources of uncertainty may have not been fully accounted for. Potential systematic errors include calibration inaccuracies of the spectrometer, wavelength drifts, and small imperfections in the iodine cell such as temperature or pressure variations. Statistical uncertainties might arise from sources such as detector sensitivity fluctuations and background light variations. In addition to this, even though several scans of each source were performed in order to improve reproducibility, small inconsistencies may still affect the overall errors and results.

Further research could include the heating of the iodine cell in order to obtain a better signal of spectral bands due to the ($v' \leftarrow 1''$) transitions which would allow for the calculations of molecular constants such as ω''_e , $\omega''_e X''_e$ and D''_e ultimately allowing for a proper study of the X and B band of iodine and their Morse potential as a function of internuclear radius. Additionally, performing the experiment using fluorescence spectroscopy instead of - or alongside - absorption would provide more precise measurements of the vibrational bands and reduce the effects of overlapping lines, improving the accuracy of the extracted molecular constants.

References

- [1] N. Krishnamurthy Simon George. Absorption spectrum of iodine vapor—an experiment. *American Journal of Physics*, pages 850–853, 1989.
- [2] J. L. Cruickshank E. L. Lewis, C. W. P. Palmer. Iodine molecular constants from absorption and laser fluorescence. *American Journal of Physics*, pages 350–356, 1994.
- [3] Fred E. Stafford. Band spectra and dissociation energies: A physical chemistry experiment. *Journal of Chemical Education*, pages 626–629, 1962.
- [4] G. Herzberg. *Molecular Spectra and Molecular Structure, Vol. I: Spectra of Diatomic Molecules*. D. Van Nostrand Company, Inc., 2nd edition, 1950.
- [5] NIST Standard Reference Data. Handbook of basic atomic spectroscopic data. <https://www.nist.gov/pml/handbook-basic-atomic-spectroscopic-data>, 2013.

- [6] John Quinn. The χ^2 test for goodness of fit, 2024. University College Dublin, Physics Labs Documentation.

Appendix

```

import numpy as np
from matplotlib import pyplot as plt
from scipy.signal import find_peaks
from scipy.optimize import curve_fit
from glob import glob # To open files
from scipy.stats import chi2 # to obtain the p-value from the chi
squared

# Size plots & labels
plt.rcParams['figure.figsize'] = (12,6)
plt.rcParams['axes.labelsize'] = 18
plt.rcParams['xtick.labelsize'] = 14
plt.rcParams['ytick.labelsize'] = 14
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['legend.fontsize'] = 14

```

Data Acquisition

```

from pydaqmx_helper.adc import ADC

myADC = ADC()
myADC.addChannels([0])

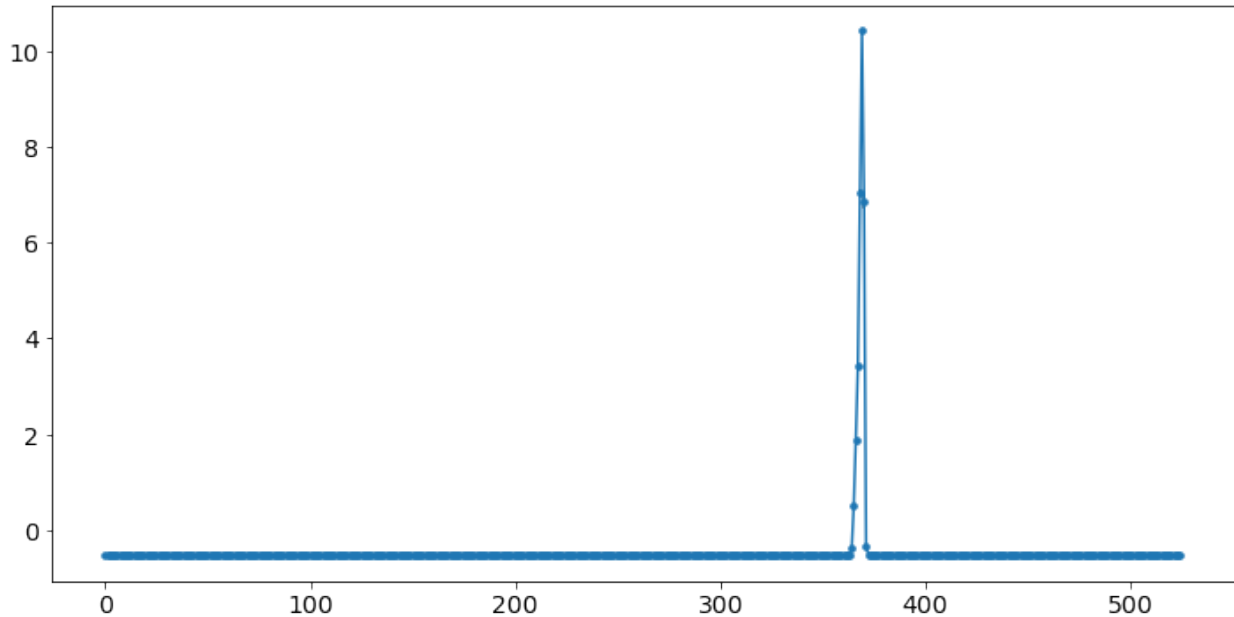
Activated Channel 0

V = np.zeros(3125)
V_err = np.zeros(3125)
for i in range(3125): # 3125 data points
    data = myADC.sampleVoltages(100, 5000)[0] # Take 100
    measurements at 5000 measurements per second as 1 data point
    V[i] = np.mean(data)
    V_err[i] = np.std(data)/np.sqrt(100) # Standard mean on the
    error

iterations = np.arange(525)
plt.errorbar(iterations, V, yerr=V_err, ecolor='grey', marker='o',
ms=3.5)
#plt.xlim(2900,3125)

<ErrorbarContainer object of 3 artists>

```



```
#np.savetxt('Downloads/Hg_650_400_5.csv', np.column_stack((V, V_err)),
header='V, V_err')
```

Calibration

Calibration (single measurement)

```
# Import calibration data
V_laser, V_laser_err = np.loadtxt('Data/_laser_650_400.csv',
unpack=True, delimiter=' ', skiprows=1)
V_Hg, V_Hg_err = np.loadtxt('Data/_Hg_650_400_0.csv', unpack=True,
delimiter=' ', skiprows=1)
V_Na, V_Na_err = np.loadtxt('Data/_Na_650_400_0.csv', unpack=True,
delimiter=' ', skiprows=1)

# Find peaks & their properties
laser_peaks, laser_props = find_peaks(V_laser, height=0)
Hg_peaks, Hg_props = find_peaks(V_Hg, height=0)
Na_peaks, Na_props = find_peaks(V_Na, height=0)

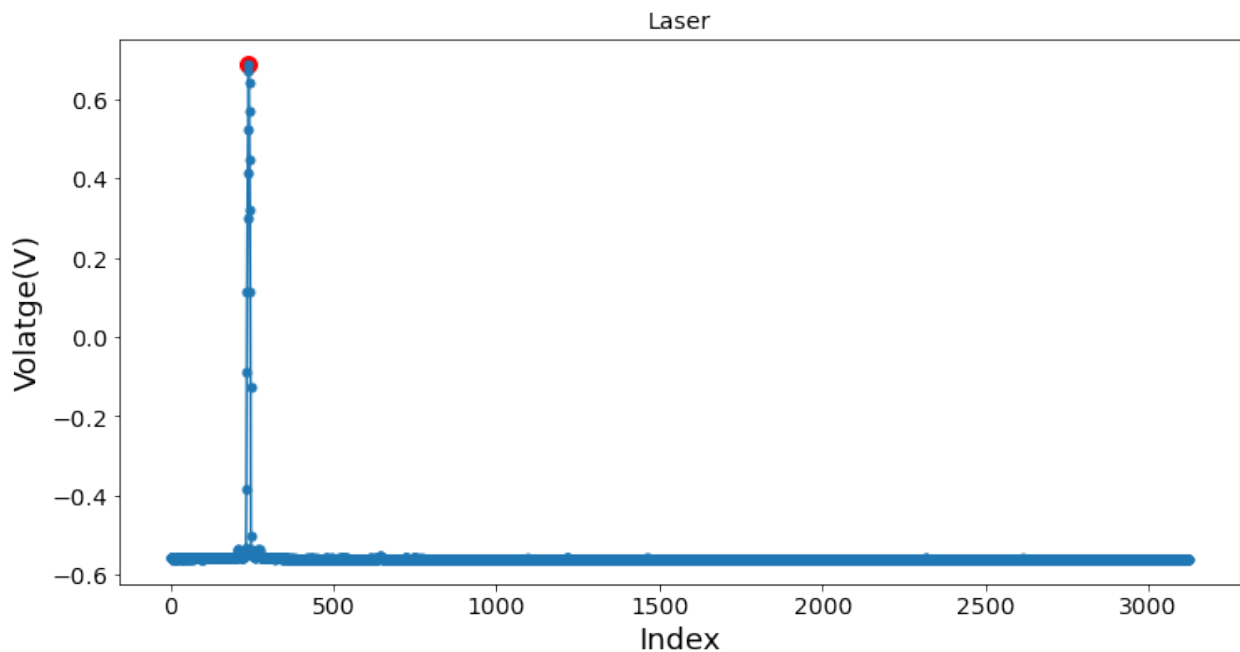
# Create dataset
dataset = [
    {"source": "Laser", "V": V_laser, "V_err": V_laser_err, "wl_th":
[632.816], "peaks": laser_peaks, "peak_heights":
laser_props["peak_heights"]},
    {"source": "Hg", "V": V_Hg, "V_err": V_Hg_err, "wl_th": [546.0735,
435.8328], "peaks": Hg_peaks, "peak_heights":
Hg_props["peak_heights"]},
```

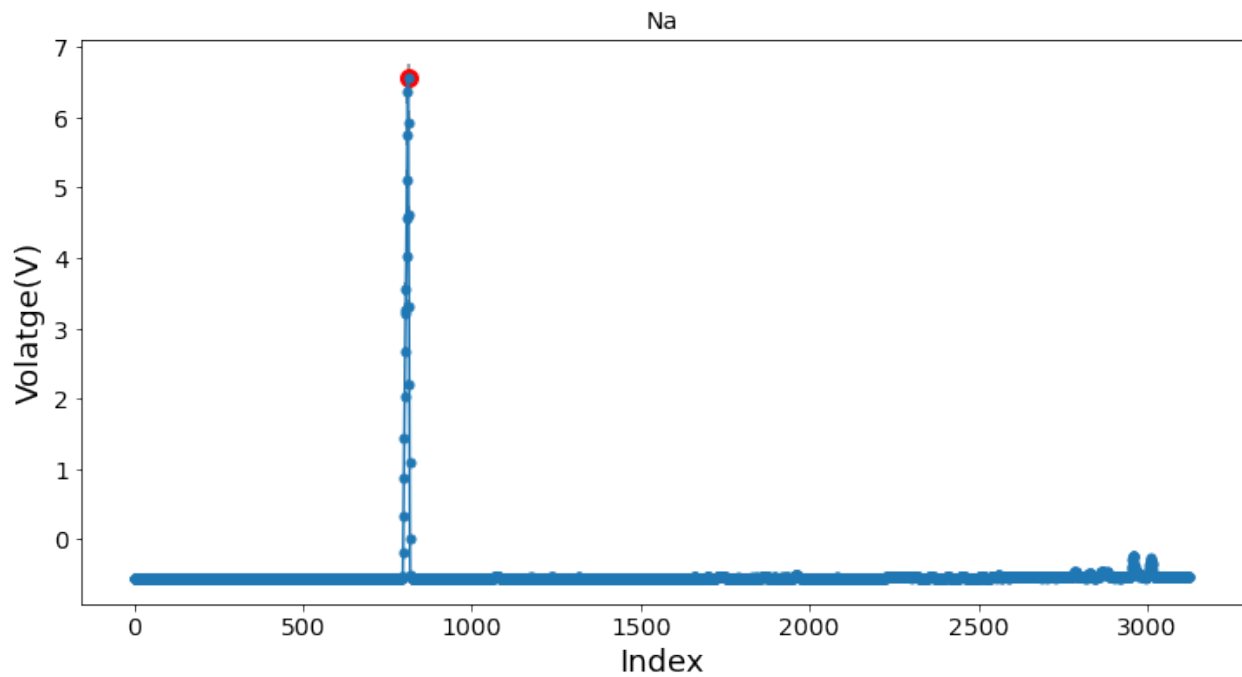
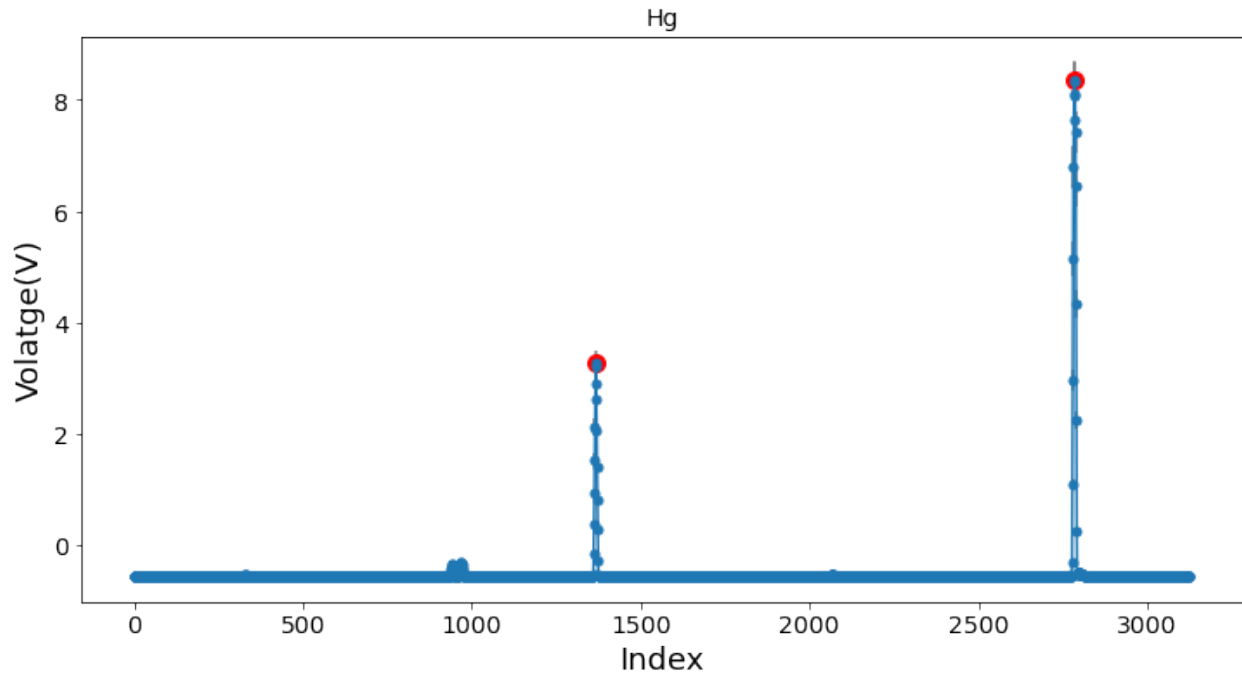
```

    {"source": "Na", "V": V_Na, "V_err": V_Na_err, "wl_th":
[588.9950], "peaks": Na_peaks, "peak_heights":
Na_props["peak_heights"]}
]

# Plot spectra of all sources
iterations = np.arange(len(V_laser))
for ds in dataset:
    plt.errorbar(iterations, ds['V'], yerr=abs(ds['V_err']),
ecolor='grey', marker='o', ms=5)
    plt.scatter(ds['peaks'], ds['peak_heights'], color='r',
marker='o', s=100, zorder=2)
    #plt.xlim(900,1000)
    #plt.ylim(-0.6, -0.3)
    plt.xlabel('Index')
    plt.ylabel('Volatge(V)')
    plt.title(ds['source'])
    plt.show()

```





```
# Define function of a straight line for best-fit-line of calibration
def f(x,m,c):
    return m*x + c

# Initialize variables to store data points of peaks
peakses, wl_ths = [], []

# Store and plot the data points
```



```

for ds in dataset:
    peakses.extend(ds['peaks'])
    wl_ths.extend(ds['wl_th'])

    plt.scatter(ds['peaks'], ds['wl_th'], label=ds['source'])

# Obtain values for best fit
popt, pcov = curve_fit(f, peakses, wl_ths)
m, c = popt
m_err, c_err = np.sqrt(np.diag(pcov))

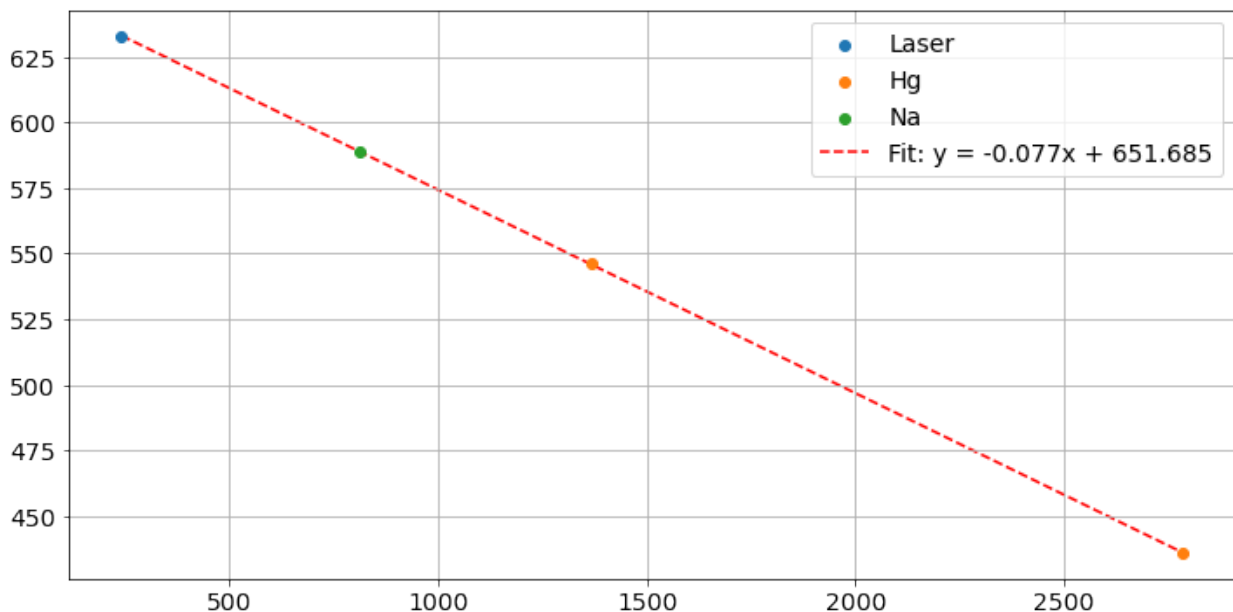
# Print them for a visual check
print(f"Best-fit line: {m:.4f}x + {c:.4f}")
print(f"Uncertainties on: m = {m_err:.4f}, c = {c_err:.4f}")

# Plot the line
xfit = np.linspace(min(peakses), max(peakses), 5000)
yfit = f(xfit, m, c)
plt.plot(xfit, yfit, 'r--', label=f"Fit: y = {m:.3f}x + {c:.3f}",
zorder=0)

plt.legend()
plt.grid()
plt.show()

```

Best-fit line: $-0.0775x + 651.6849$
Uncertainties on: $m = 0.0002$, $c = 0.2758$



Multimeasurements

For Mercury (Hg), two additional spectral lines appeared toward the end of the spectrum, but only in some of the measurements. The reason they only showed up in some runs and not in all is likely due to starting the recording slightly later in those cases and/or the motor skipping certain wavelengths.

An attempt was made to include these lines in the calibration, but it was not successful. This is because, since they appeared only under specific conditions, incorporating them (without properly accounting for their true positions in the runs where they were not visible) would have introduced the bias of only choosing runs which had the same set of conditions. This bias would reduce the errors to unrealistically small values, ultimately changing the calibration line.

In order to avoid the inclusion of these lines, only data points before the mark 3000 were considered. This restriction does not affect any other spectral line from any other source.

```
# Define a function to find the average position of all the peaks in
all of the calibration sources
def get_peak_pos(paths, plot=False):
    file_list = glob(paths)
    peaks_pos = []

    for file in file_list:
        V, V_err = np.loadtxt(file, unpack=True, delimiter=' ',
skiprows=1)
        pos, props = find_peaks(V[:3000], height=0) # Identify peaks
        peaks_pos.append(pos)

        # Visual check
        if plot==True:
            iterations = np.arange(len(V))
            plt.errorbar(iterations, V, yerr=abs(V_err), marker='o',
ms=3.5, ecol='grey')
            plt.scatter(pos, props['peak_heights'], color='red', s=80,
zorder=1)
            plt.xlim(3000,)
            plt.ylim(-0.6, 0)
            plt.show()
    return np.mean(peaks_pos, axis=0), np.std(peaks_pos, axis=0)

# Import calibration data
laser_paths = "Data/laser*.csv"
Hg_paths = "Data/Hg*.csv"
Na_paths = "Data/Na*.csv"

# Obtain position of peaks
laser_pos, laser_pos_err = get_peak_pos(laser_paths)
Hg_pos, Hg_pos_err = get_peak_pos(Hg_paths)
Na_pos, Na_pos_err = get_peak_pos(Na_paths)
```

```

# Create dataset
dataset = {
    "Laser": {"peaks": laser_pos, "peaks_err": laser_pos_err, "wl_th":
[632.816]},
    "Hg": {"peaks": Hg_pos, "peaks_err": Hg_pos_err, "wl_th":
[546.0735, 435.8328]},
    "Na": {"peaks": Na_pos, "peaks_err": Na_pos_err, "wl_th":
[589.5924, 588.9950]}
}

# Define function of a straight line for best-fit-line of calibration
def f(x,m,c):
    return m*x + c

# Define function to test goodness of fit
def chi_squared(measured, expected, measured_err, n_params):
    chi_sq = np.sum(((measured-expected) / measured_err)**2)
    dof = len(measured) - n_params
    p_val = chi2.sf(chi_sq, dof)
    return chi_sq, p_val

plt.rcParams['figure.figsize'] = (5, 6)

# Initialize variables to store data points of peaks
peakses, peakses_err, wl_ths = [], [], []
# Store and plot the data points
for ds in dataset:
    peakses.extend(dataset[ds]['peaks'])
    peakses_err.extend(dataset[ds]['peaks_err'])
    wl_ths.extend(dataset[ds]['wl_th'])

    plt.errorbar(dataset[ds]['wl_th'], dataset[ds]['peaks'],
yerr=dataset[ds]['peaks_err'], label=ds, marker='o', ms=7, ls='None',
capsize=5)

# Obtain values for best fit
popt, pcov = curve_fit(f, wl_ths, peakses, sigma=peakses_err,
absolute_sigma=True)
m, c = popt
m_err, c_err = np.sqrt(np.diag(pcov))
# Print them for a visual check
print(f"Best-fit line: {m:.4f}x + {c:.4f}")
print(f"Uncertainties on: m = {m_err:.4f}, c = {c_err:.4f}")

# Plot the line
xfit = np.linspace(650, 400, 500)
yfit = f(xfit, m, c)
plt.plot(xfit, yfit, color='grey', ls='--', label="Best-fit line",
zorder=0)

```

```
plt.xlabel('Wavelengths (nm)')
plt.ylabel('Counts')
plt.legend()
plt.grid()
plt.savefig("calibration_line.png", dpi=300, bbox_inches='tight')
plt.show()
```

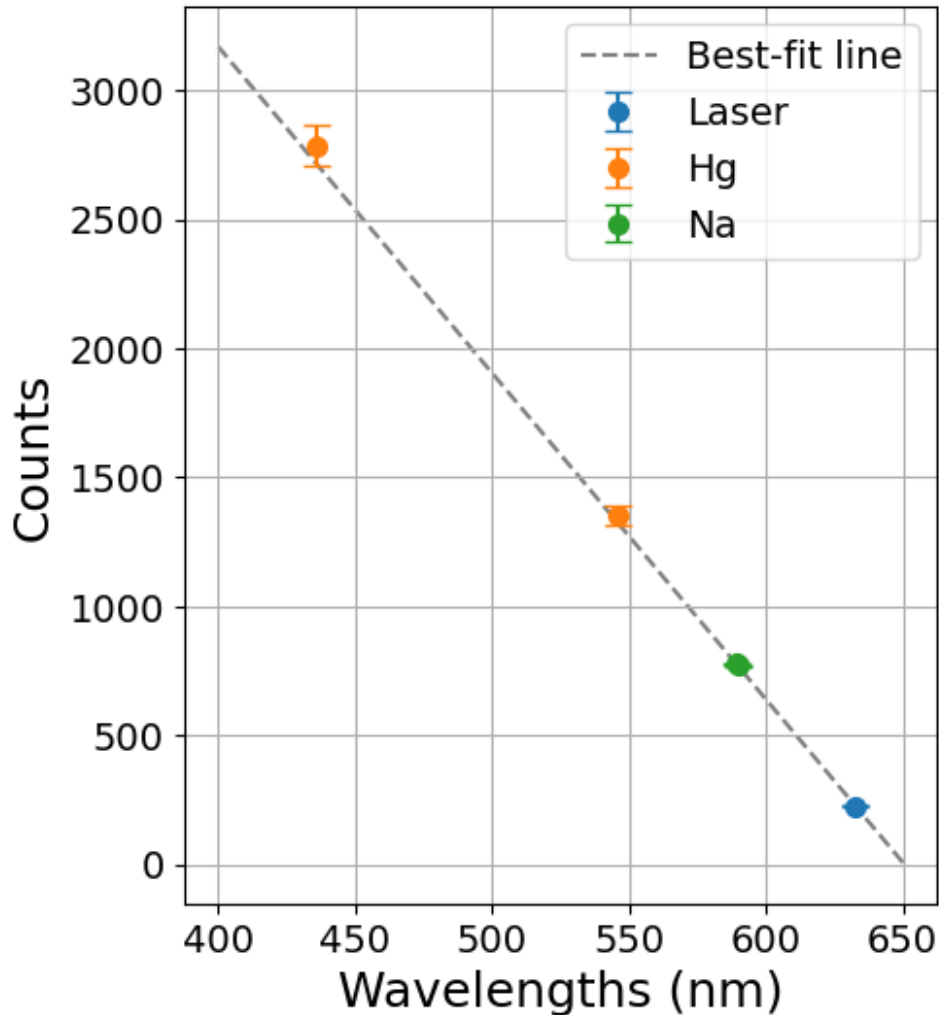
Use the chi_squared function

```
chi_sq, p_val = chi_squared(peakses, f(np.array(wl_ths), m, c),
peakses_err, len(popt))
print(f"Chi-squared = {chi_sq:.2f}, p-value = {p_val:.3f}")
```

```
plt.rcParams['figure.figsize'] = (12,6)
```

Best-fit line: $-12.6549x + 8231.7026$

Uncertainties on: $m = 0.1109$, $c = 68.6586$



Chi-squared = 1.67, p-value = 0.644

Calibration (including 2 extra Hg lines)

```
# Define a function to find the average position of all the peaks in
all of the calibration sources
def get_peak_pos(paths, plot=False, extra=False):
    file_list = glob(paths)
    peaks_pos = []

    for file in file_list:
        V, V_err = np.loadtxt(file, unpack=True, delimiter=' ',
skiprows=1)
        if extra:
            pos, props = find_peaks(V[3000:], height=-0.3) #
Identify peaks
            pos+=3000
        else: pos, props = find_peaks(V[:3000], height=0)
        peaks_pos.append(pos)

        # Visual check
        if plot==True:
            iterations = np.arange(len(V))
            plt.errorbar(iterations, V, yerr=abs(V_err), marker='o',
ms=3.5, ecolor='grey')
            plt.scatter(pos, props['peak_heights'], color='red', s=80,
zorder=1)
            plt.show()
    return np.mean(peaks_pos, axis=0), np.std(peaks_pos, axis=0)

# Import calibration data
laser_paths = "Data/laser*.csv"
Hg_paths = "Data/Hg*.csv"
Hg_extra_paths = "Data/Hg_extra*.csv"
Na_paths = "Data/Na*.csv"

# Obtain position of peaks
laser_pos, laser_pos_err = get_peak_pos(laser_paths)
Hg_pos, Hg_pos_err = get_peak_pos(Hg_paths)
Hg_extra_pos, Hg_extra_pos_err = get_peak_pos(Hg_extra_paths,
extra=True)
Na_pos, Na_pos_err = get_peak_pos(Na_paths)

# Create dataset
dataset = {
    "Laser": {"peaks": laser_pos, "peaks_err": laser_pos_err, "wl_th":
[632.816]},
    "Hg": {"peaks": Hg_pos, "peaks_err": Hg_pos_err, "wl_th":
[546.0735, 435.8328]},
    "Hg_extra": {"peaks": Hg_extra_pos, "peaks_err": Hg_extra_pos_err,
"wl_th": [404.6563, 398.3931]},
    "Na": {"peaks": Na_pos, "peaks_err": Na_pos_err, "wl_th":
```

```

[589.5924, 588.9950]]
}

# Define function of a straight line for best-fit-line of calibration
def f(x,m,c):
    return m*x + c

# Initialize variables to store data points of peaks
peakses, peakses_err, wl_ths = [], [], []

# Store and plot the data points
for ds in dataset:
    peakses.extend(dataset[ds]['peaks'])
    peakses_err.extend(dataset[ds]['peaks_err'])
    wl_ths.extend(dataset[ds]['wl_th'])

    plt.errorbar(dataset[ds]['wl_th'], dataset[ds]['peaks'],
yerr=dataset[ds]['peaks_err'], label=ds, marker='o', ms=5, ls='None',
capsize=3)

# Obtain values for best fit
popt, pcov = curve_fit(f, wl_ths, peakses, sigma=peakses_err,
absolute_sigma=True)
m, c = popt
m_err, c_err = np.sqrt(np.diag(pcov))

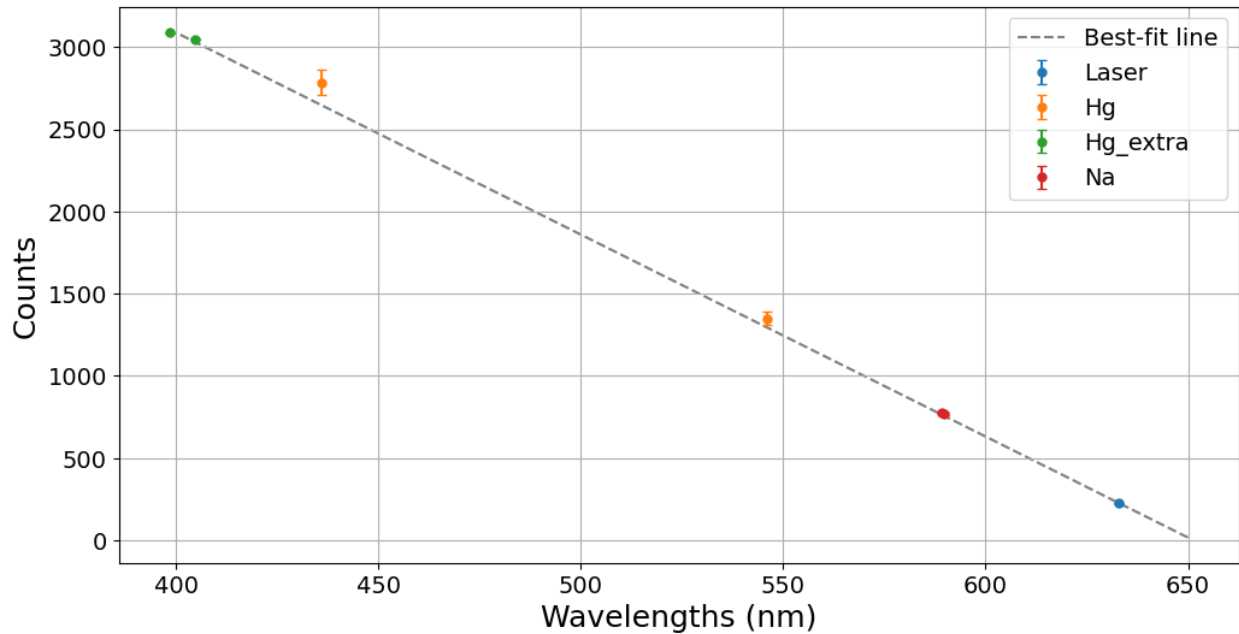
# Print them for a visual check
print(f"Best-fit line: {m:.4f}x + {c:.4f}")
print(f"Uncertainties on: m = {m_err:.4f}, c = {c_err:.4f}")

# Plot the line
xfit = np.linspace(650, 400, 500)
yfit = f(xfit, m, c)
plt.plot(xfit, yfit, color='grey', ls='--', label="Best-fit line",
zorder=0)

plt.xlabel('Wavelengths (nm)')
plt.ylabel('Counts')
plt.legend()
plt.grid()
plt.show()

Best-fit line: -12.2823x + 8000.6072
Uncertainties on: m = 0.0186, c = 10.3335

```



Data Analysis

Wavelength offset

Same way recording of calibration might not have started at exactly 650.0 nm, these recording might not be exact either. Some peaks from iodine will be identified in order to properly align the wavelengths with the recordings and a pair of without-iodine recording that follows a similar trend (since they changed slightly) and has the peak around the same point in the x-axis.

Check for best fit overlapping

```
iterations = np.arange(3125)
wavelengths = (iterations - c) / m

# Import data for tungsten lamp with iodine
tungsen_w_paths = 'Data/tungsen_w_*.csv'
tungsen_w_files = glob(tungsen_w_paths)

# Import data for tungsten lamp without iodine
tungsen_wo_paths = 'Data/tungsen_wo_*.csv'
tungsen_wo_files = glob(tungsen_wo_paths)

# Rotate through w_files while plotting all of wo_files to find a match
for j in range(len(tungsen_w_files)):
    file_w = tungsen_w_files[j]

    V_w, V_w_err = np.loadtxt(file_w, unpack=True, delimiter=' ',
```

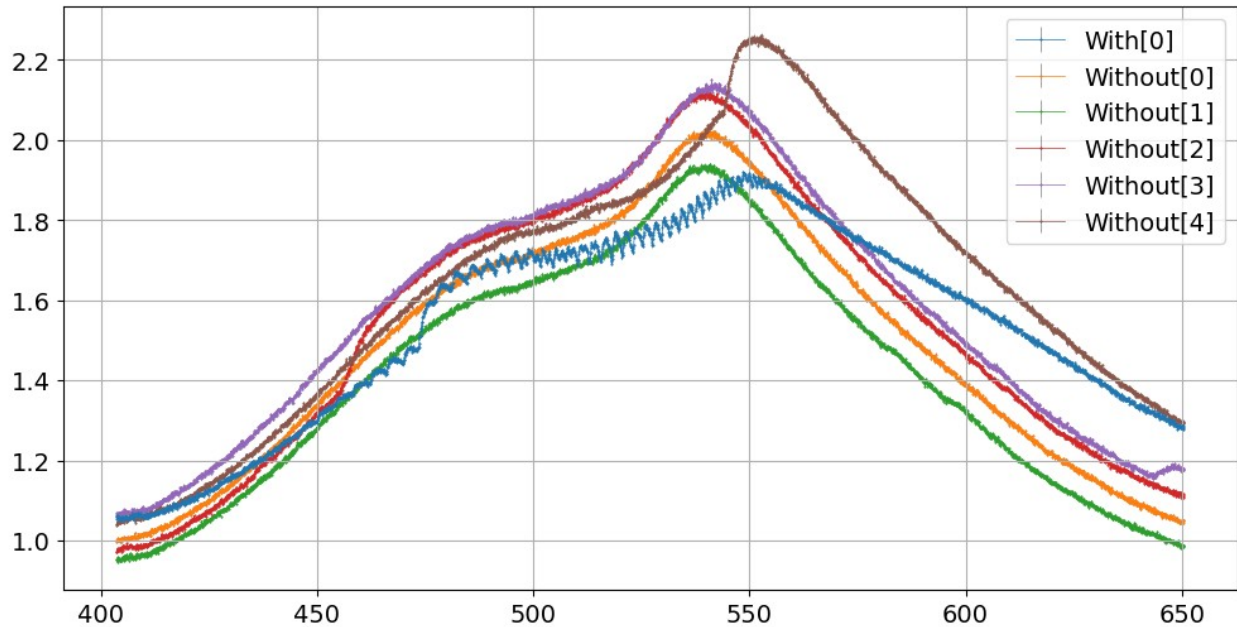
```

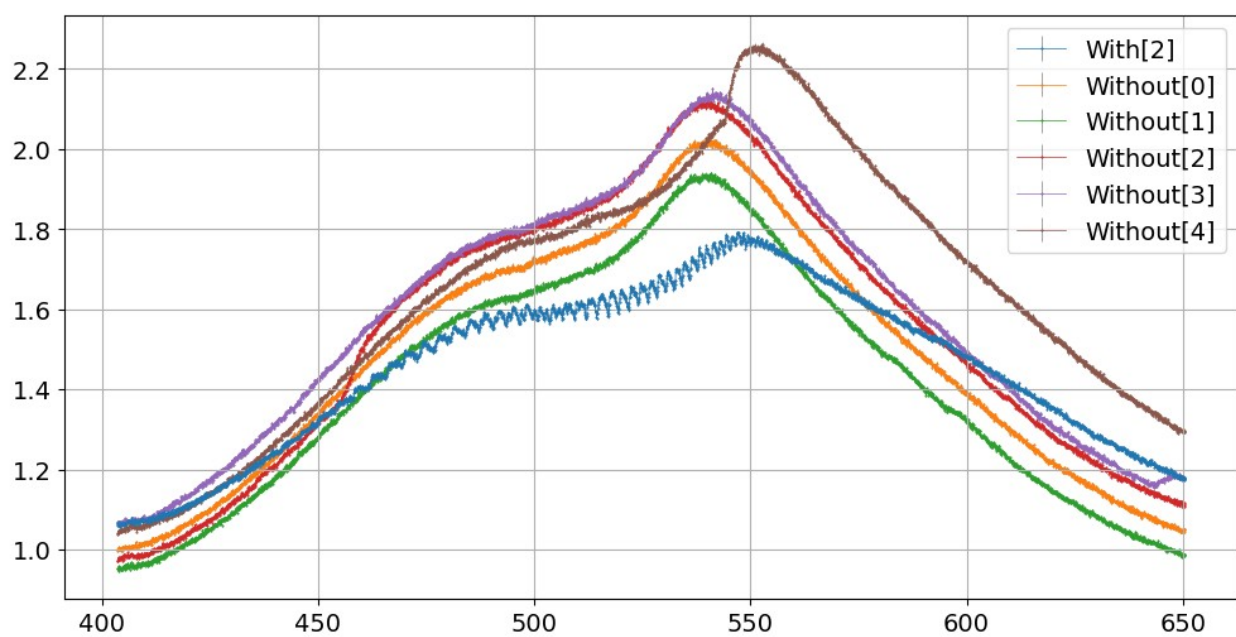
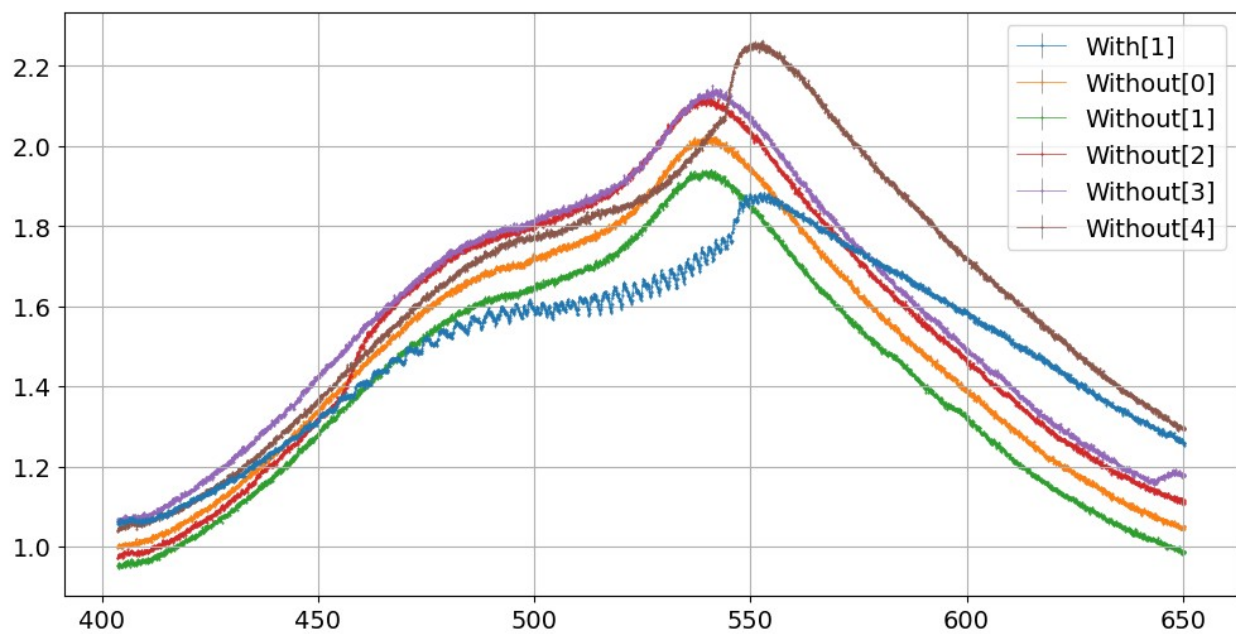
skiprows=1)
plt.errorbar(wavelengths[::1], V_w, yerr=V_w_err, ecolor='grey',
marker='o', ms=0.5, linewidth=0.5, label=f'With{[j]}')

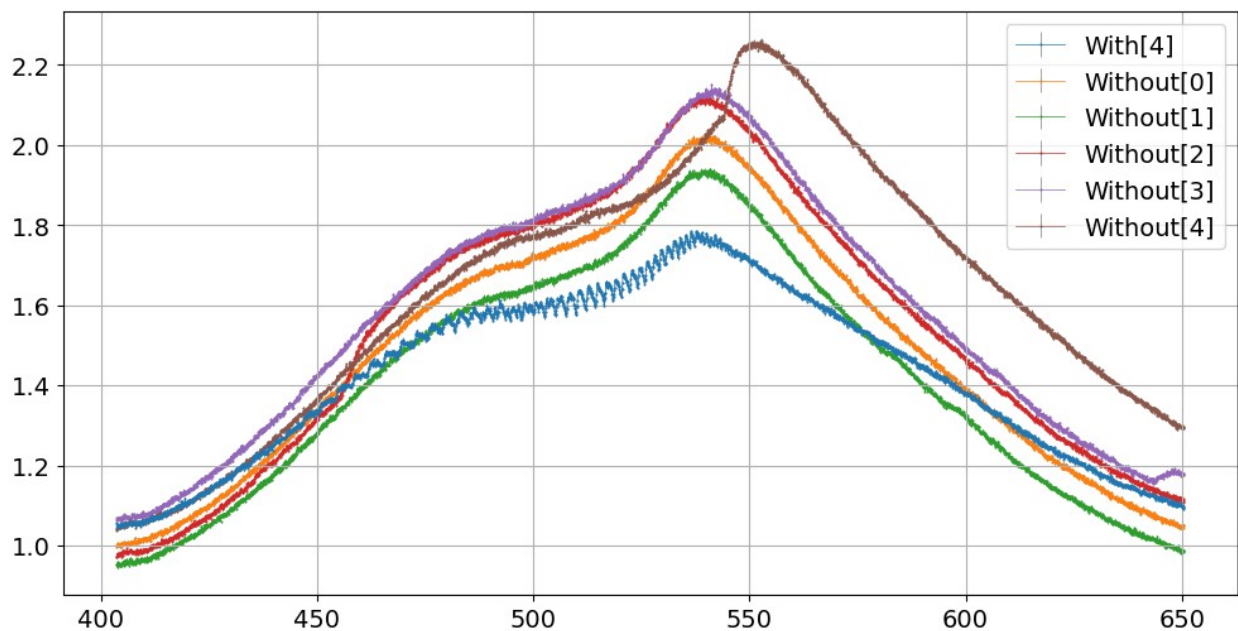
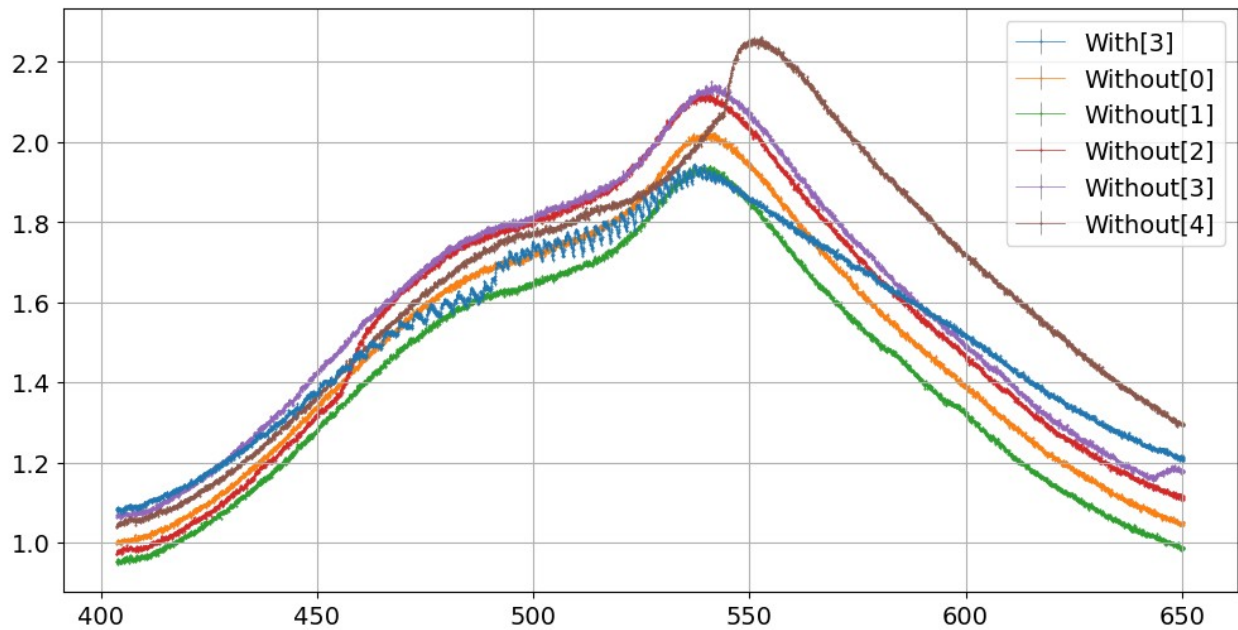
for i in range(len(tungsen_wo_files)):
    file_wo = tungsen_wo_files[i]
    V_wo, V_wo_err = np.loadtxt(file_wo, unpack=True, delimiter='
', skiprows=1)
    plt.errorbar(wavelengths[::1], V_wo, yerr=V_wo_err,
ecolor='grey', marker='o', ms=0.5, linewidth=0.5, zorder=0,
label=f'Without{[i]}')

#plt.xlim(520,550)
plt.legend()
plt.grid()
plt.show()

```







Calculate and apply offset

```
iterations = np.arange(3125)
wavelengths = (iterations - c) / m

# Plot curve with iodine
file_w = tungsen_w_files[1] # Alternatively, 4
V_w, V_w_err = np.loadtxt(file_w, unpack=True, delimiter=' ',
skiprows=1)
plt.errorbar(wavelengths, V_w, yerr=V_w_err, ecolor='grey',
marker='o', ms=0.5, linewidth=0.5, label='With iodine')
```

```

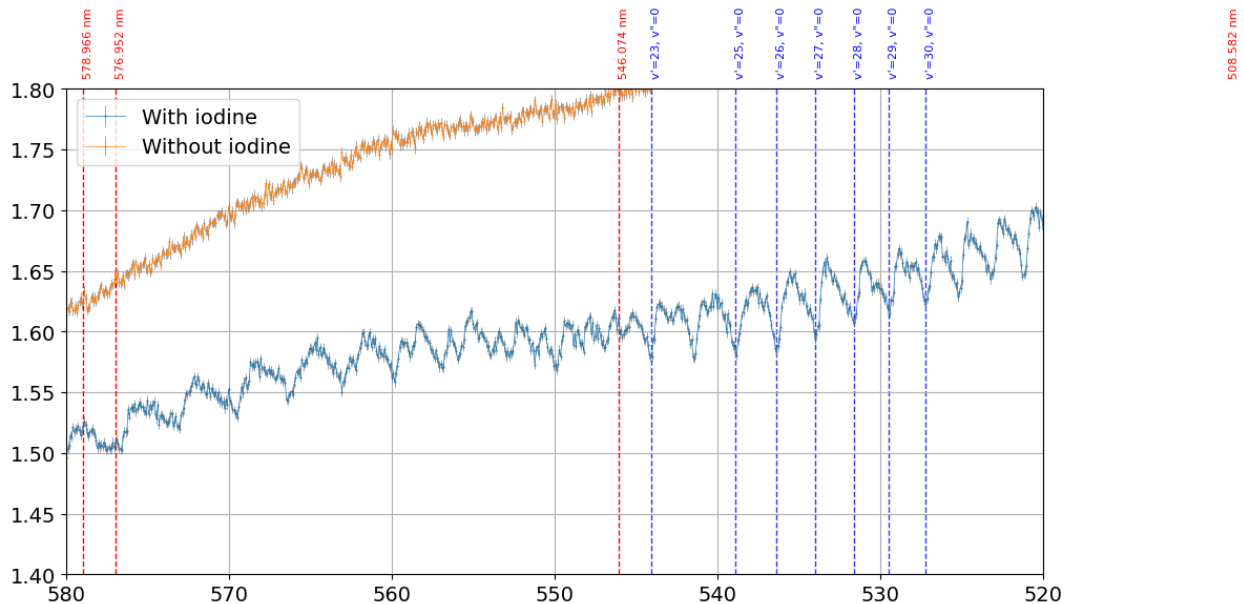
# Plot curve without iodine
file_wo = tungsen_wo_files[4] # Alternatively, along with 4 above, 0
V_wo, V_wo_err = np.loadtxt(file_wo, unpack=True, delimiter=' ',
skiprows=1)
plt.errorbar(wavelengths, V_wo, yerr=V_wo_err, ecolor='grey',
marker='o', ms=0.5, linewidth=0.5, label='Without iodine')

# Plot vertical red lines from paper for reference
for line in [578.966, 576.952, 546.074, 508.582]:
    plt.axvline(line, linestyle='--', color='r', linewidth=1)
    plt.text(line, 1.81, f"{line:} nm", rotation=90, color='r',
    fontsize=8)

# Plot vertical blue lines indicating key spots in the spectra for
reference
for line, name in zip([544.1, 538.9, 536.38, 534, 531.6, 529.5,
527.2], [23, 25, 26, 27, 28, 29, 30]):
    #line = (line_pos - c) / m # Turn into wavelength
    plt.axvline(line, linestyle='--', color='b', linewidth=1,
zorder=0, alpha=0.8)
    plt.text(line, 1.81, f"v'={name}, v\"=0", rotation=90, color='b',
    fontsize=8)

plt.gca().invert_xaxis()
plt.xlim(580,520)
plt.ylim(1.4,1.8)
plt.legend()
plt.grid()
plt.show()

```



From "Iodine molecular constants from absorption and laser fluorescence", Fig 2 known spectral lines of other elements were plotted as red lines. For example, Hg shows a line 546.074 nm which coincides with $v'=25, v''=0$. However the lines at quantum numbers 23, 24, 25, etc. can be identified by their shape and compared to the ones in the plot above after which one can realise that they do not match. Line 23 was identified as the first big dip after what is labeled as 'section 2' in the paper, line 26 could be described the one right before what had turned into a series of 2 consecutive peaks (section 2) combines back to 1 single sharp peak. With lines 27 and 28 being the precedent of 2 peaks with quite recognisable shape due to their decrease.

The spectra both with and without iodine (since they match in shape implying the difference between starting the recording and starting the motor was similar between both runs) will now be calibrated by readjusting the spectral lines $v'=26$ through 30 with the values in "Band spectra and Dissociation Energies"

```
# Bands wavelengths
th_wl = [546.82, 543.47, 541.18, 539.89, 536.87]
obs_wl = [536.38, 534, 531.6, 529.5, 527.2]
# To compensate for manually choosing the line wavelength
iter_uncertainty = 1 # iteration uncertainty
iter_to_nm = abs( m**(-1) ) # nm per iteration
wl_err = iter_uncertainty * iter_to_nm # nm

# Calculate and apply offset
differences = [i-j for i,j in zip(th_wl,obs_wl)]
wl_offset = np.mean(differences)
print(f"Wavelength offset: {wl_offset:.4f}")
wl_offset_err = np.std(differences)/np.sqrt(len(differences))
print(f'Statistical uncertainty: {wl_offset_err:.4f} nm')
```

```

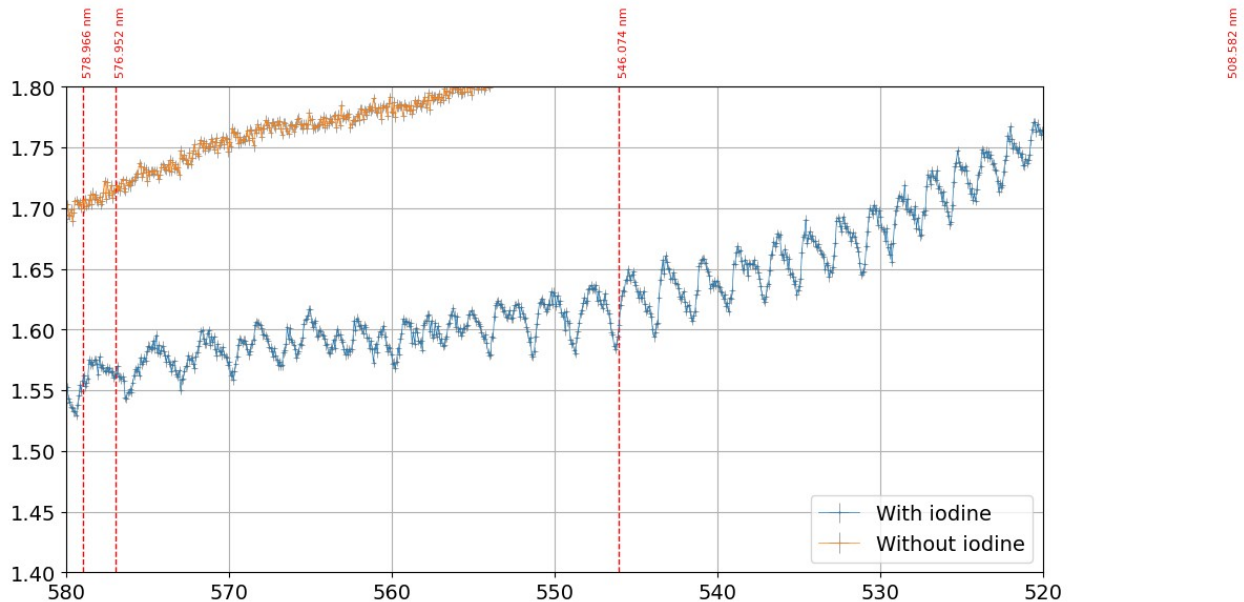
# Apply and plot
wavelengths = (iterations - c) / m
wavelengths += wl_offset
plt.errorbar(wavelengths, V_w, xerr=wl_offset_err, yerr=V_w_err,
marker='o', ms=0.5, linewidth=0.5, ecolor='grey', label='With iodine')
plt.errorbar(wavelengths, V_wo, xerr=wl_offset_err, yerr=V_wo_err,
marker='o', ms=0.5, linewidth=0.5, ecolor='grey', label='Without
iodine')

# Plot vertical red lines from paper for reference
for line in [578.966, 576.952, 546.074, 508.582]:
    plt.axvline(line, linestyle='--', color='r', linewidth=1)
    plt.text(line , 1.81, f"{line:} nm", rotation=90, color='r',
    fontsize=8)

plt.gca().invert_xaxis()
plt.xlim(580,520)
plt.ylim(1.4,1.8)
plt.legend()
plt.grid()
plt.show()

```

Wavelength offset: 9.9100
Statistical uncertainty: 0.1867 nm



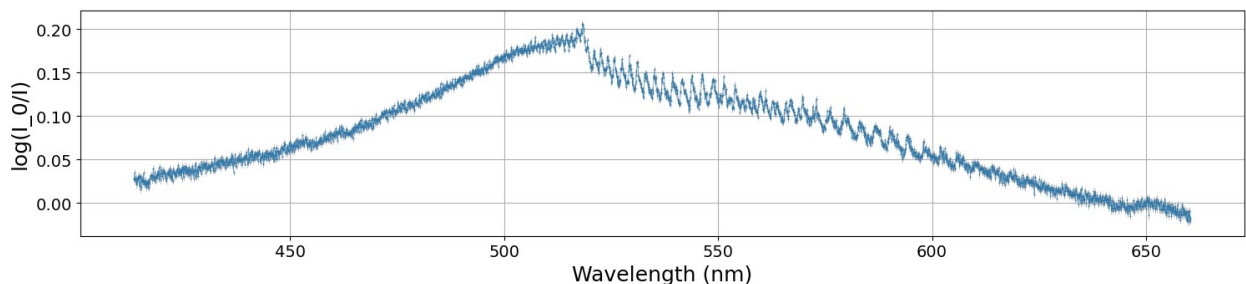
As a visual check, the Hg lines around 579 and 577 nm are 11 quantum numbers away from the line at 546 nm, just like in the paper

Logarithmic plot

```
plt.rcParams['figure.figsize'] = (18,3.5)

# Plot the logarithm
A = np.log(V_wo/V_w)
A_err = np.sqrt((V_wo_err / V_wo)**2 + (V_w_err / V_w)**2)

plt.errorbar(wavelengths, A, xerr=wl_offset_err, yerr=A_err,
ecolor='grey', marker='o', ms=0.5, linewidth=0.5, alpha=0.7)
plt.xlabel("Wavelength (nm)")
plt.ylabel("log(I0/I)")
plt.grid()
plt.savefig("log_plot.png", dpi=300, bbox_inches='tight')
plt.show()
plt.rcParams['figure.figsize'] = (12,6)
```



1st difference vs Quantum number

```
# Define the wavelength window
mask = ((wavelengths >= 520) & (wavelengths <= 555)) | ((wavelengths
>= 565) & (wavelengths <= 620))
wl_region = wavelengths[mask]
A_region = A[mask]

# Find minima in this region
minima, props = find_peaks(A_region, prominence=0.02, distance=5)
minima_pos = wl_region[minima]
minima_depths = A_region[minima]

# Plot
plt.errorbar(wavelengths, A, xerr=wl_offset_err, yerr=A_err,
ecolor='grey', marker='o', ms=0.5, linewidth=0.2, label='Data')
plt.plot(minima_pos, minima_depths, color='r', ms=3, marker='o',
linestyle='None', label='Spectral lines quantum numbers')
[plt.text(x, y+0.01, f"v'={23+i}", rotation=90, color='grey',
fontsize=8)
for i, (x, y) in enumerate(zip(minima_pos[-17:], minima_depths[-
17:])))
[plt.text(x, y+0.01, f"v'={18-(len(minima_pos[:-17])-1-i)}",
```

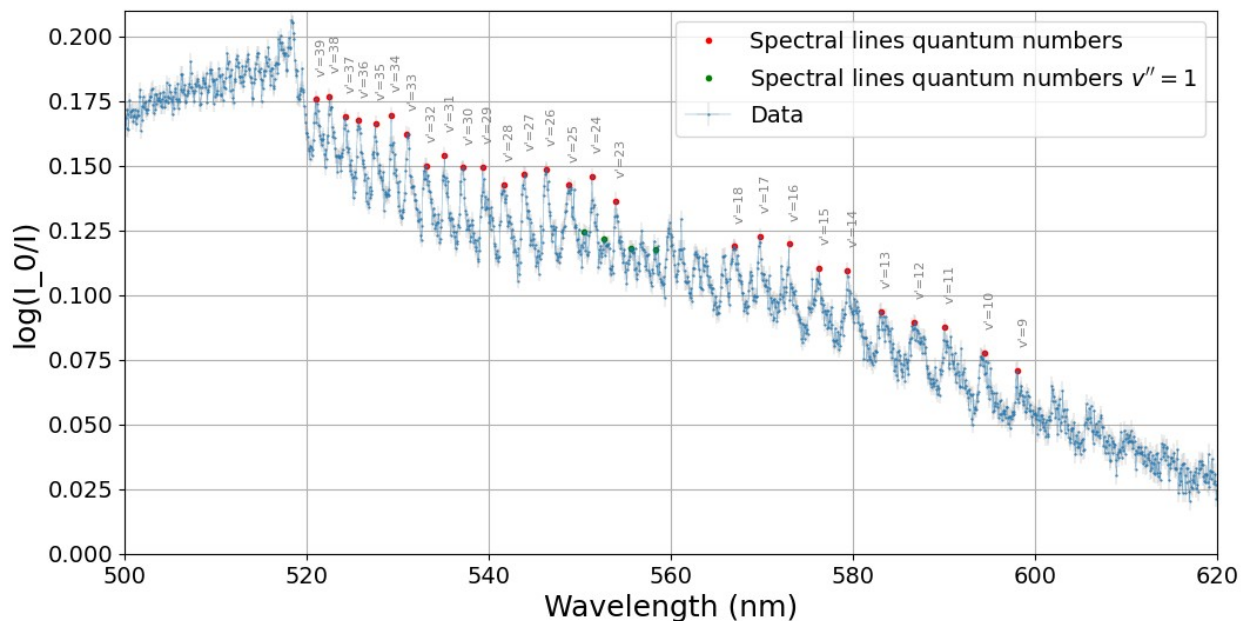
```

rotation=90, color='grey', fontsize=8)
    for i, (x, y) in enumerate(zip(minima_pos[:-17], minima_depths[:-17]))]

# Bands for  $v''=1$ 
minima_g = np.where(mask)[0][minima]
minima_v1 = [minima_g[-15]-22, minima_g[-16]-16, minima_g[-17]-21,
minima_g[-17]-56] # 27, 26, 25, 24
plt.plot(wavelengths[minima_v1], A[minima_v1], color='oran', ms=3,
marker='o', linestyle='None', label='Spectral lines quantum numbers
 $v''=1$ ')

plt.xlim(500, 620)
plt.ylim(0, 0.21)
plt.xlabel("Wavelength (nm)")
plt.ylabel("log(I_0/I)")
plt.legend()
plt.grid()
plt.show()

```



The idea of including the bands from $v''=1$ was abandoned since they had to be found manually, the signal wasn't great and there is only very little of them that can be recognised by a human eye

```

# Manual correction of some peaks that were not properly detected - it
happens more in the 2nd region since peaks in 1st are more sharp and
clear
minima_pos[1] = wl_region[minima[1]+4]
minima_pos[2] = wl_region[minima[2]-3]
minima_pos[5] = wl_region[minima[5]-4]

```


No bands past 39 or 9 can be detected. This is due to the high amount of noise and/or the lost of precision that taking measurements there would suppose

```
plt.rcParams['figure.figsize'] = (5, 7)

# Quantum numbers
v_numbers1 = np.arange(9, 19)
v_numbers2 = np.arange(23, 40)

# Grab the middle point for plotting the differences
v_mid1 = [(v_numbers1[i]+v_numbers1[i+1])/2 for i in
range(len(v_numbers1)-1)]
v_mid2 = [(v_numbers2[i]+v_numbers2[i+1])/2 for i in
range(len(v_numbers2)-1)]

# Get the wavenumbers in cm-1
wavenumbers = 1e7 / minima_pos # convert nm to cm-1
wavenumbers_err = 1e7 * wl_err / (minima_pos**2) #
wl_err=minima_pos_err as it is unchanged through: wavelengths,
wl_region, minima_pos

# Calculate the differences and its errors
differences1 = np.diff(wavenumbers[:len(v_numbers1)])
differences2 = np.diff(wavenumbers[len(v_numbers1):])
differences1_err = np.sqrt(wavenumbers_err[1:len(v_numbers1)]**2 +
wavenumbers_err[:len(v_numbers1)-1]**2) # vi+1 - vi ->
sqrt(Δvi+1**2 + Δvi**2)
differences2_err = np.sqrt(wavenumbers_err[len(v_numbers1)+1:]**2 +
wavenumbers_err[len(v_numbers1):-1]**2)

# Join together
v_ = np.array(v_mid1 + v_mid2)
DG = np.array(list(differences1) + list(differences2))
DG_err = np.array(list(differences1_err) + list(differences2_err))

# Get fit parameters
popt, pcov = curve_fit(f, v_, DG, sigma=DG_err, absolute_sigma=True)
m_fit, c_fit = popt
m_err, c_err = np.sqrt(np.diag(pcov))
print(f"Best fit: y = ({m_fit:.4f} ± {m_err:.4f})x + ({c_fit:.4f} ±
{c_err:.4f})")

# Plot fit line
x_fit = np.linspace(min(v_), max(v_), 200)
y_fit = f(x_fit, m_fit, c_fit)
plt.plot(x_fit, y_fit, 'r-', label="Best-fit line")

# Plot data with error bars
plt.errorbar(v_, DG, yerr=DG_err, fmt='o', capsize=3, label="Data with
errors", zorder=0)
```



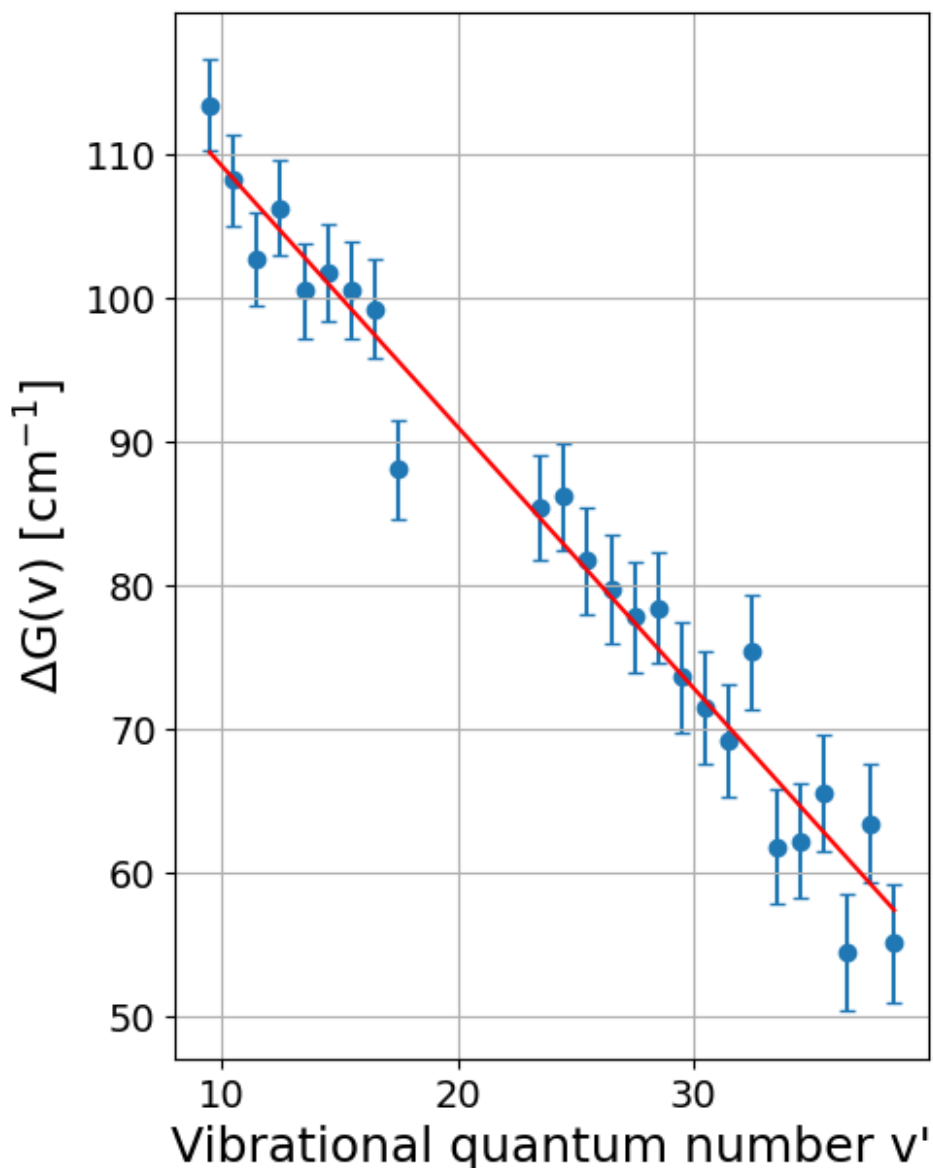
```

plt.xlabel("Vibrational quantum number v'")
plt.ylabel("ΔG(v) [cm$^{-1}$]")
#plt.legend()
plt.grid()
plt.savefig("BS_plot.png", dpi=300, bbox_inches='tight')
plt.show()

# Use the chi_squared function
chi_sq, p_val = chi_squared(DG, f(np.array(v_), m_fit, c_fit), DG_err,
len(popt))
print(f"Chi-squared = {chi_sq:.2f}, p-value = {p_val:.3f}")

plt.rcParams['figure.figsize'] = (12, 6)
Best fit:  $y = (-1.8161 \pm 0.0773)x + (127.3506 \pm 1.9262)$ 

```



Chi-squared = 19.32, p-value = 0.683

Obtaining the physical parameters

Graphical results

```

WeXe = -m_fit/2
WeXe_err = m_err / 2
print(f"WeXe = {WeXe:.4f} ± {WeXe_err:.4f} cm-1")

We = c_fit + WeXe*1    # Since We is the intercept at v=-1
We_err = np.sqrt(c_err**2 + WeXe_err**2)
print(f"We = {We:.4f} ± {We_err:.4f} cm-1")

```

```

v_m = -c_fit/m_fit
v_m_err = np.sqrt( (c_err / m_fit)**2 + (c_fit * m_err /
m_fit**2)**2 )
print(f"v_m = {v_m:.4f} ± {v_m_err:.4f}")

De = v_m * We / 2 # Area of triangle = base * height / 2
De_err = np.sqrt( (v_m * c_err / 2)**2 + (c_fit * v_m_err / 2)**2 )
print(f"De = {De:.4f} ± {De_err:.4f} cm^-1")

D0 = De - 1/2*We + 1/4*WeXe # G(0) = 1/2*We - 1/4*WeXe + ...
D0_err = np.sqrt(De_err**2 + (1/2*We_err)**2 + (1/4*WeXe_err)**2)
print(f"D0 = {D0:.4f} ± {D0_err:.4f} cm^-1")

mu = 1.053e-22 # g
c_speed = 2.998e10 # cm/s
ke = (2*np.pi*c_speed*We)**2 * mu
ke_err = (2*np.pi*c_speed)**2 * 2*We * We_err * mu
print(f"ke = {ke:.4g} ± {ke_err:.4g} dyn/cm")

WeXe = 0.9081 ± 0.0386 cm^-1
We = 128.2587 ± 1.9266 cm^-1
v_m = 70.1217 ± 3.1666
De = 4496.8611 ± 212.6454 cm^-1
D0 = 4432.9587 ± 212.6476 cm^-1
ke = 6.146e+04 ± 1847 dyn/cm

np.diff(Gv)

array([131.89097742, 133.70711398, 135.52325054, 137.33938711,
       139.15552367, 140.97166023, 142.78779679, 144.60393335])

h = 6.62607015e-27
beta = We * np.sqrt(2*mu*c_speed/(De*h))
re = 3.016e-8
r = np.linspace(0.5*re, 5*re, 500)
U = De * (1 - np.exp(-beta * (r - re)))**2
plt.plot(r, U, linewidth=2.5, zorder=0)

# Plot
G0 = We*0.5 + WeXe*1/4
plt.axhline(G0, linestyle='--', color='purple')
plt.text(1.5e-7, G0+100, f"G(0)", color='purple', fontsize=12)

v_list = [10,20,30,40,50]
for v in v_list:
    Gv = We*(v+0.5) - WeXe*(v+0.5)**2
    plt.axhline(Gv, linestyle='--', color='grey', linewidth=1,
zorder=0)
    plt.text(1.5e-7, Gv+100, f"G({v})", color='grey', fontsize=12)

plt.axhline(De, linestyle='--', color='r', label=f'$D_e={De:.2f}$',

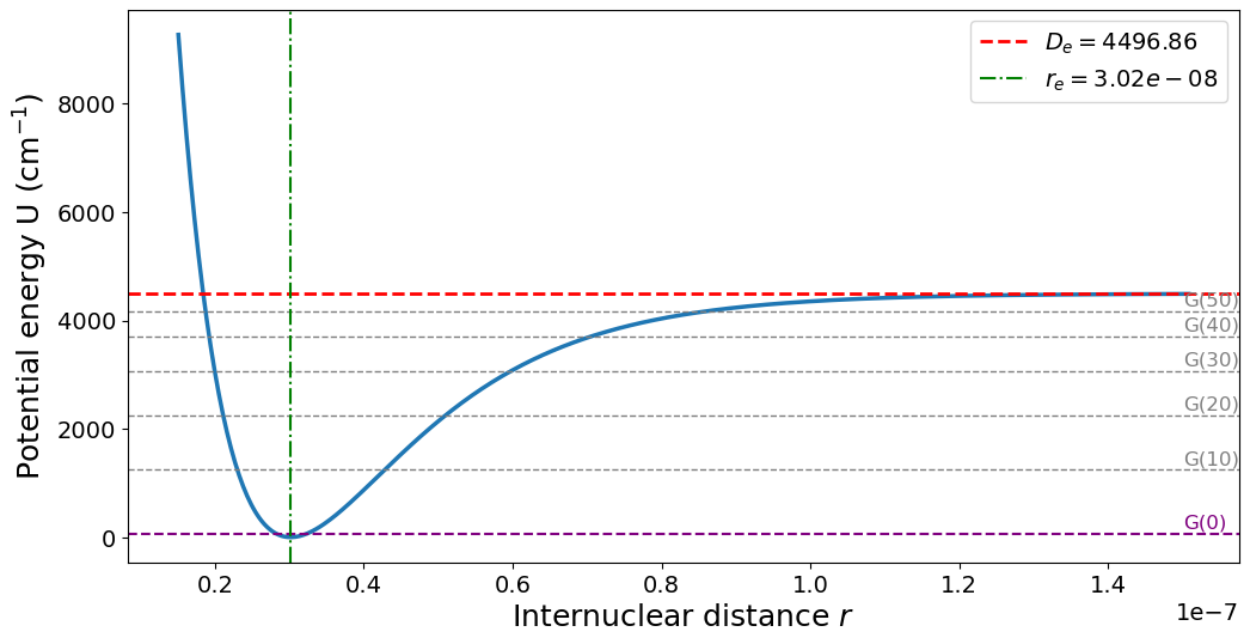
```

```

linewidth=2)
plt.axvline(re, linestyle='dashdot', color='g', label=f'$r_e={re:.3g}$')

plt.xlabel("Internuclear distance $r$")
plt.ylabel("Potential energy U (cm$^{-1}$)")
#plt.grid()
plt.legend()
plt.savefig("Morse_potential.png", dpi=300, bbox_inches='tight')
plt.show()

```



```

# Numerical results

D2G = np.diff(DG) # Second differences
D2G_err = np.sqrt(DG_err[1:]**2 + DG_err[:-1]**2)

WeXe = abs(np.mean(D2G)) / 2
WeXe_err = np.sqrt(np.mean(D2G_err**2)) / (2*len(D2G_err))
print(f"WeXe (numerical) = {WeXe:.4f} ± {WeXe_err:.4f} cm-1")

We = np.mean(DG + 2 * WeXe * v_)
We_err = np.sqrt(np.mean(DG_err**2)/len(DG) +
np.mean((2*v_*WeXe_err)**2)/len(v_))
print(f"We (numerical) = {We:.4f} ± {We_err:.4f} cm-1")

Xe = WeXe/We
Xe_err = np.sqrt((WeXe_err / We)**2 + (WeXe * We_err / We**2)**2 )
print(f"Xe = {Xe:.4g} ± {Xe_err:.4g}")

v_m = 1/(2*Xe) - 1

```

```

v_m_err = Xe_err / (2 * Xe**2)
print(f"v_m (numerical) = {v_m:.4f} ± {v_m_err:.4f}")

De = We*(v_m+1/2) - WeXe*(v_m+1/2)**2 # De = G(v_m)
De_err = np.sqrt( ((v_m+1/2)*We_err)**2 + ((v_m+1/2)**2*WeXe_err)**2)
print(f"De = {De:.4f} ± {De_err:.4f} cm^-1")

D0 = De - 1/2*We + 1/4*WeXe # G(0) = 1/2*We - 1/4*WeXe + ...
D0_err = np.sqrt(De_err**2 + (1/2*We_err)**2 + (1/4*WeXe_err)**2)
print(f"D0 = {D0:.4f} ± {D0_err:.4f} cm^-1")

mu = 1.053e-22 # g
c_speed = 2.998e10 # cm/s
ke = (2*np.pi*c_speed*We)**2 * mu
ke_err = (2*np.pi*c_speed)**2 * 2*We * We_err * mu
print(f"ke = {ke:.4g} ± {ke_err:.4g} dyn/cm")

WeXe (numerical) = 1.2138 ± 0.1089 cm^-1
We (numerical) = 142.4535 ± 1.3656 cm^-1
Xe = 0.008521 ± 0.000769
v_m (numerical) = 57.6815 ± 5.2961
De = 4179.3919 ± 377.1907 cm^-1
D0 = 4108.4686 ± 377.1913 cm^-1
ke = 7.582e+04 ± 1454 dyn/cm

```